



Republic of Tunisia
Ministry of Higher Education and Scientific Research
University of Jendouba
Higher Institute of Computer Science of Kef



End-of-Study Project Report

Presented at

Higher Institute of Computer Science of Kef

In partial fulfillment of the requirements for the
National bachelor's degree in computer engineering

Specialization:

Network and Computer Systems Engineering
By

Rabeb Dhahri

Implementing a SIEM solution with open-source tools

Presented on: 30/05/2025

In front of the jury of commission:

President: Mr. Haythem Hayouni

Reviewer: Mr. Radhouen Aloui

Professional supervisor: Mr. Souhaïel Hamza

Academic supervisor: Mrs. Marwa Sammouda

Academic Year: 2024-2025

Acknowledgment

I take this opportunity to express my deep sense of gratitude and sincere thanks to all who helped me to complete the work successfully. My first and foremost thanks goes to God Almighty who showered in immense blessings on my effort.

I wish to express my sincere thanks to Mr. SOUHAIEL HAMZA, my workplace mentor, and to my academic supervisor Mrs. MARWA SAMMOUDA for providing me with all the necessary facilities and support.

I also want to give special thanks to Mrs. SHIRAZ OUERTANI for all her help and support.

Likewise, I thank my parents, all my family and friends near and dear ones who directly and indirectly contribute to the success of this work.

At the finish line, I am grateful for the personal and professional growth this project afforded me, and I appreciate everyone who supported me along the way.

RABEB DHAHRI

Table of Contents

General introduction	9
Chapter 1: The state of art.....	10
1.1 Introduction	11
1.2 Company presentation.....	11
1.3 Project statement	11
1.3.1 Problem statement	11
1.3.2 Proposed solution.....	12
1.4 Theoretical approach of SIEM solutions	12
1.4.1 SIEM definition.....	12
1.4.2 SIEM architecture	13
1.4.3 SIEM functions	14
1.4.4 SIEM benefits.....	15
1.4.5 Most common SIEM vendors	16
1.5 Intrusion detection system.....	16
1.5.1 Intrusion detection system definition.....	16
1.5.2 Functional architecture of an intrusion detection system.....	17
1.5.3 Most popular intrusion detection systems tools	18
1.6 Log forwarding.....	18
1.6.1 Log forwarding definition	18
1.6.2 Log forwarders functions.....	18
1.6.3 Most common log forwarders.....	19
1.7 Conclusion	20
Chapter 2: Implementation of the SIEM solution	21
2.1 Introduction	22
2.2 Global architecture of the SIEM solution	22
2.2.1 Suricata	23
2.2.2 Syslog.....	24
2.2.3 Splunk	25

2.3	System setup tools	28
2.3.1	VMware workstation	28
2.3.2	Ubuntu virtual machine	29
2.4	Implementation	29
2.4.1	Installation and configuration of Suricata.....	29
2.4.2	Installation and configuration of Syslog	30
2.4.3	Installation and configuration of Splunk.....	32
2.5	Conclusion	38
Chapter 3: Test and validation of the SIEM solution.....		39
3.1	Introduction	40
3.2	Attack strategy	40
3.2.1	Attack tools.....	40
3.3	Attack execution	42
3.3.1	Reconnaissance.....	42
3.3.2	Brute force	43
3.3.3	Privilege Escalation.....	45
3.3.4	Persistence	45
3.3.5	Tracks cover	45
3.4	Detection and alerting.....	46
3.5	Prevention.....	47
3.5.1	Password strengthening	48
3.5.2	User account audit and cleanup	48
3.5.3	IP blocking.....	48
3.5.4	SSH configuration hardening and port modification.....	49
3.5.5	Fail2ban Deployment.....	50
3.6	Conclusion	50
General conclusion		51
Bibliography		52

Table of figures

Figure 1.1: Company logo	11
Figure 1.2 : SIEM architecture.....	14
Figure 1.3: Integration of an IDS	17
Figure 2.1: Architecture of the SIEM solution.....	22
Figure 2.2: Suricata logo	23
Figure 2.3: Syslog	24
Figure 2.4: Splunk logo.....	25
Figure 2.5: Splunk architecture	26
Figure 2.6: Splunk indexer cluster architecture	28
Figure 2.7: VMware Workstation logo	29
Figure 2.8: Ubuntu logo	29
Figure 2.9: Suricata installation	30
Figure 2.10: Suricata status	30
Figure 2.11: Network interface configuration.....	30
Figure 2.12: Syslog installation	30
Figure 2.13: Syslog status	31
Figure 2.14: Syslog configuration for system logs	31
Figure 2.15: Syslog configuration for suricata logs.....	32
Figure 2.16: Splunk installation.....	32
Figure 2.17: Splunk data input configuration	33
Figure 2.18: Splunk alerts	33
Figure 2.19: Triggered alerts	34
Figure 2.20: Splunk table dashboard	34
Figure 2.21: Data volume dashboard	35
Figure 2.22: Enable clustering	35
Figure 2.23: Master node configuration.....	36
Figure 2.24: Enable peer node	37
Figure 2.25: Peer node configuration.....	37
Figure 2.26: Peer nodes.....	38
Figure 3.1: Kali Linux logo	40
Figure 3.2: Nmap logo	41
Figure 3.3: Hydra logo	41
Figure 3.4: Zenmap	42
Figure 3.5: Port scan	42
Figure 3.6: A customized password list	43

Figure 3.7: Brute-force attack	44
Figure 3.8: SSH login	44
Figure 3.9: Sudo privileges check.....	45
Figure 3.10: Creation of a new user.....	45
Figure 3.11: Log files remove.....	45
Figure 3.12: Ping detection	46
Figure 3.13: Nmap detection.....	46
Figure 3.14: Brute-force detection.....	46
Figure 3.15: Creation of a new user alert.....	46
Figure 3.16: Creation of a new group alert	46
Figure 3.17: Splunk mobile app alerts	47
Figure 3.18: Change password.....	48
Figure 3.19: Reviewing user accounts	48
Figure 3.20: Delete suspicious users.....	48
Figure 3.21: Extract attacker IP address	48
Figure 3.22: Block attacker IP address	49
Figure 3.23: Disable SSH root login.....	49
Figure 3.24: Turn off password-based authentication.....	49
Figure 3.25: Enable key-based authentication	49
Figure 3.26: Change SSH port.....	49
Figure 3.27: Allow traffic on the new SSH port	50
Figure 3.28: Fail2ban installation	50

List of tables

Table 1.1: Difference between the most common SIEM solutions.....	16
Table 1.2: Difference between the most popular IDS tools	18
Table 1.3: Difference between the most common log forwarders	19

List of abbreviations

SIEM: Security Information and Event Management

SIM: Security Information Management

SEM: Security Event Management

SMS: Short Message Service

SNMP: Simple Network Management Protocol

IP: Internet Protocol

SaaS: Software as a Service

CNI : Centre National de l'Informatique

IT : Information Technology

ISO: International Organization for Standardization

ML: Machine Learning

EPS: Events Per Second

IDS: Intrusion Detection System

URL: Uniform Resource Locator

TLS: Transport Layer Security

SSH: Secure Shell

IPS: Intrusion Prevention System

NSM: Network Security Monitoring

FPC: Full Packet Capture

PCAP: Packet Capture

AWS: Amazon Web Services

IoC: Indicators of Compromise

VM: Virtual Machine

OS: Operating System

GUI: Graphical User Interface

AI: Artificial Intelligence

TCP: Transmission Control Protocol

App: Application

General introduction

During the past few years, cybersecurity risks affecting organizations have been evolving rapidly, driven by more sophisticated and dangerous threats from advanced cybercriminals. As these threats become more and more complex, detecting and mitigating them becomes a real challenge for security Teams. Therefore, vendors launched systems that provide both SIM (Security Information Management) and SEM (Security Event Management) to create SIEM (Security Information and Event Management) solutions.

This is where SIEM solutions come in as a critical tool to fight against cybercrimes by collecting logs from multiple sources, detecting threats and analyzing data. This helps security teams to easily manage logs and identify suspicious activities faster with a centralized platform, offering security events correlation and real-time monitoring.

The main goal of a SIEM solution is to manage risks by detecting threats early and responding faster to security incidents. Also, it generates alerts and notifications that can be delivered via email, SMS or other communication channels.

This project will dive more into SIEM solutions by exploring its core features, components, architecture and integration with other security technologies. It's divided into three chapters:

- The first chapter “The state of art” presents the project context, explains SIEM concepts, and justifies the selection of the implemented tools.
- The second chapter “Implementation of the SIEM solution” details the technical setup of the SIEM environment. It explains the architecture of the solution, the roles of each component, and walks through their installation and configuration.
- The third chapter “Test and validation of the SIEM solution” covers a simulated cyberattack to test the SIEM setup and shows how threats were detected and prevented.

The following chapters will give a deep understanding of how SIEM systems work and how they win to provide the organizations' security.

Chapter 1

The state of art

1.1 Introduction

In this chapter, we will present in the first place, the company where we carried out our internship, then, we will provide an overview of the project, including the problem statement and the proposed solution. Finally, we will present the theoretical framework that supports the solution.

1.2 Company presentation

The National Center of Informatics is a public administrative body with legal personality and financial autonomy. It was established on December 30, 1975. CNI is an institution under the authority of the Ministry of Communication Technologies. It operates in the fields of IT and communication technologies and is certified in accordance with ISO 9001- 2015 [1].

The figure 1.1 shows the logo of the National Center of Informatics [1]:



Figure 1.1: Company logo

1.3 Project statement

The following section presents the problem statement and the proposed solution to give a clear understanding of the project.

1.3.1 Problem statement

As cyberattacks grow in complexity and scale, the limitations of our existing security architecture rooted in standalone, legacy tools have become increasingly apparent. Our organization currently utilizes a combination of traditional security solutions, including firewalls, antivirus software, and intrusion detection systems, to safeguard our IT infrastructure. While these tools provide essential baseline protection, they operate in isolation and generate uncorrelated logs and alerts. This fragmented approach makes it challenging for our security team to effectively detect and prioritize genuine threats amidst a high volume of false positives.

The absence of a centralized logging and monitoring solution significantly limits our ability to gain a holistic view of security events across the environment. As a result, threat detection and incident response processes are largely manual, time-consuming, and prone to oversight. Furthermore, the lack of automated correlation, real-time alerting, and integrated compliance

reporting impedes our capacity to conduct timely investigations and demonstrate regulatory compliance.

Given the increasing sophistication and volume of modern cyber threats, it is evident that our current security architecture must evolve. Continuing to rely solely on standalone security tools leaves the organization vulnerable to undetected threats, inefficient response workflows, and potential compliance risks.

1.3.2 Proposed solution

To overcome the limits of old, isolated security solutions, installing a Security Information and Event Management (SIEM) system has emerged as an essential move toward building a more robust and responsive security architecture. SIEM systems provide an integrated platform for collecting and normalizing log and event data from a range of sources, such as servers, firewalls, endpoints, databases, and network devices. SIEM consolidates this data, giving security experts a real-time, unified view of the organization's threat landscape.

One of the core strengths of a SIEM system is its ability to correlate disparate events across the environment to identify patterns that may indicate malicious activity. Rather than relying on isolated alerts from individual tools, SIEM applies predefined correlation rules, behavioral analytics, and threat intelligence feeds to detect anomalies and sophisticated attack chains that would otherwise go unnoticed.

In summary, by delivering centralized visibility, advanced threat detection, and compliance reporting, a SIEM solution transforms our organization's security operations from reactive to proactive empowering security teams to detect, respond to, and mitigate threats more efficiently and effectively.

1.4 Theoretical approach of SIEM solutions

This section examines the fundamental theories and concepts underlying SIEM solutions.

1.4.1 SIEM definition

Security information and event management, or SIEM, is a security solution that helps organizations recognize and address potential security threats and vulnerabilities before they have a chance to disrupt business operations [2].

SIEM solutions use rules and statistical correlations to turn log entries, and events from security systems, into actionable information. This information can help security teams detect threats in real time, manage incident response, perform forensic investigation on past security incidents, and prepare audits for compliance purposes [3].

The original SIEM platforms were log management tools. They combined security information management (SIM) and security event management (SEM) functions. These platforms enabled real-time monitoring and analysis of security-related events [2].

SIM: Security information management (SIM) refers to the collection of log files and storage in a central repository for later analysis. SIM is therefore also referred to as log management. SIM solutions are often agent-based with software running on the servers and computers being monitored. These relay log and other security-related information to a central SIM server. Here, system administrators can log into a console and run security reports, graphs and charts in real time [4].

SEM: Security event management (SEM) is the identifying, gathering, monitoring, evaluating, correlating and monitoring system events and alerts. In a sense, SEM is an improvement of SIM, though the two are seen as distinct areas of security management. Just like SIM, the data is usually relayed from the host computer to a central repository using SNMP, syslog and other communication protocols. The centralized repository ensures events and alerts are kept in a forensically sound and secure storage [4].

1.4.2 SIEM architecture

SIEM architecture may vary by vendor; however, generally, essential components comprise the SIEM engine. The essential components of a SIEM are as follows:

- **Log source:** SIEM systems are based on logs from servers, network devices, sensors, and applications. These offer unprocessed data, such as system events, traffic patterns, and authentication attempts. Threat detection would be unattached to important activity due to the absence of extensive log sources.
- **Log collection:** Logs are collected from sources and normalized for processing by agents, forwarders, and protocols (e.g., Syslog). Efficient collection reduces latency and enables real-time analysis, creating the foundation for all SIEM activities.
- **Log storage:** Logs are saved in databases, data lakes, or indexed systems. This component strikes a compromise between immediate access for investigations and low-cost long-term retention for compliance and forensics.
- **SIEM engine:** To identify threats, the core analytical layer employs correlation rules, real-time monitoring, and statistical models.

- **Presentation layer:** This layer converts data into meaningful insights. Dashboards display risks, reports summarize compliance status, and consoles allow analysts to explore alarms.

Figure 1.2 will give a clear view of SIEM architecture [5]:

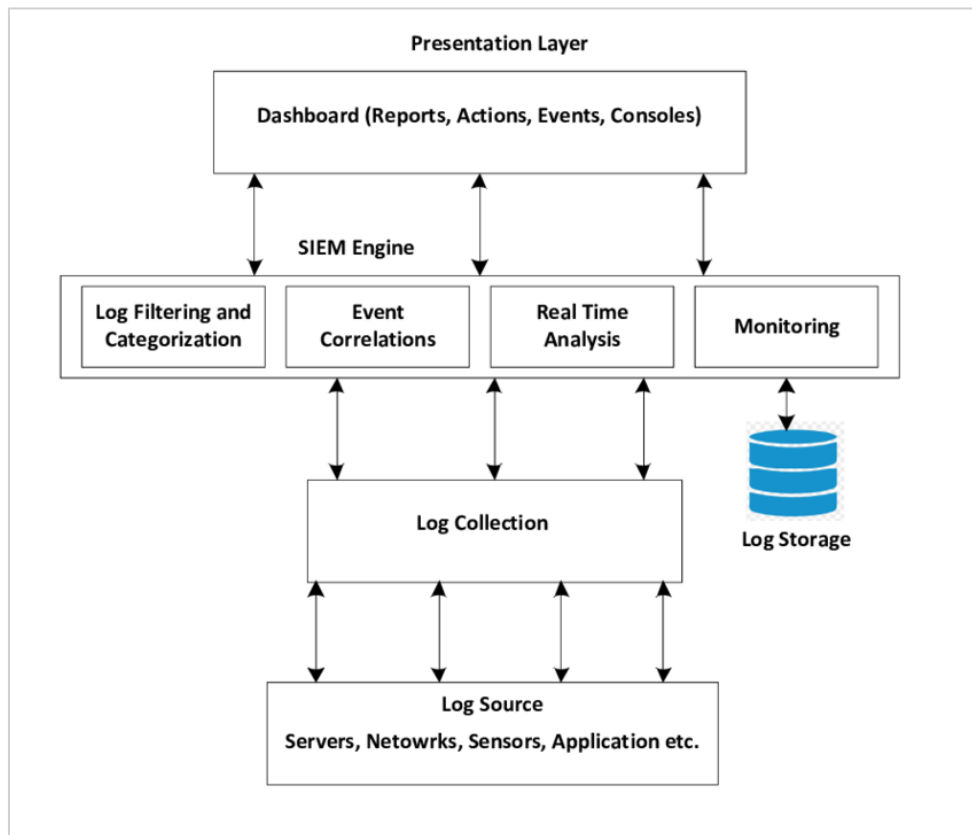


Figure 1.2 : SIEM architecture

1.4.3 SIEM functions

SIEM performs multiple functions that operate in a series of steps:

1.4.3.1 Data sources

The foundation of any SIEM system is the data it collects. This includes logs from things like firewalls, servers, routers, and intrusion detection systems. The more diverse and comprehensive your data sources, the stronger your overall security setup will be [6].

1.4.3.2 Data collection

Once you've got your data sources lined up, the next step is pulling in the information. Data collection is usually done with agents or collector tools that grab logs from devices and systems across your network. Depending on your needs, this data can be collected in real-time or at set intervals [6].

1.4.3.3 Data normalization

Not all the data you collect will come in the same format. SIEM systems need to normalize this data to make sure everything is consistent. This step is key for making it easier to analyze and correlate data from different sources, ensuring you're getting an accurate picture of what's going on [6].

1.4.3.4 Data correlation

This is the heart of SIEM systems. Correlation helps you spot patterns that could indicate security issues.

For example, a bunch of failed login attempts from different locations could point to a brute-force attack. Correlating related events allows SIEM systems to reduce noise and help security teams focus on real threats [6].

1.4.3.5 Event and incident management

Once data is correlated, it's analyzed to create alerts for the security team. A strong SIEM system offers both automated responses—like blocking a suspicious IP—and manual incident management features, helping security personnel act quickly when something's off [6].

1.4.3.6 Reporting and dashboards

Having a real-time dashboard that shows what's happening in your security environment is critical. Detailed reports help you analyze trends, track attack vectors, and find system weaknesses.

Dashboards can be customized to focus on the risks most relevant to your organization, so your security team can stay ahead of emerging threats [6].

1.4.3.7 Storage and retention

Security data needs to be stored securely for future use and compliance. SIEM systems often use high-performance databases, but as data volumes increase, many are turning into cloud-based or distributed storage solutions for the scalability they offer [6].

1.4.4 SIEM benefits

SIEM tools offer many benefits that can help strengthen an organization's overall security posture, including:

- A central view of potential threats
- Real-time threat identification and response
- Advanced threat intelligence
- Regulatory compliance auditing and reporting
- Greater transparency monitoring users, applications, and devices [7].

1.4.5 Most common SIEM vendors

There are numerous SIEM vendors that provide the same functionalities such as log management, threat detection and event correlation but each one differs in specific features and aspects.

The following table 1.1 highlights these differences:

Table 1.1: Difference between the most common SIEM solutions

Feature	Splunk	Exabeam	IBM QRadar
Deployment	On-premises, cloud, hybrid	Cloud-first	On-premises, cloud
Ease of use	User-friendly interface	User-friendly interface	Requires technical expertise
Scalability	Highly scalable	Scales with cloud	Moderate scalability
Analysis capacity	Strong search power	ML-driven insights	Deep log analysis
Cost	High with scale	Mid-range pricing	Expensive for EPS
Benefits	Most powerful analytics	Best automation	Detects a wide range of threats
Limits	Moderate learning curve	Less customizable	Complex upgrades

To wrap up, all the SIEM vendors possess the same core functionalities, but they are different in scalability, ease of use, analysis capacity and cost.

Splunk proved to be the most suitable choice. It has the most balanced and robust feature set across all critical areas. It enables flexible deployment (on-premises, cloud, or hybrid), has an easy-to-use interface, and is extremely scalable, making it appropriate for both small teams and large companies. Splunk is particularly notable for its robust search and analysis capabilities, which enable faster and more effective threat identification and response. It also offers more robust analytics with less restriction on customization and usability than Exabeam and IBM QRadar. While it may be more expensive, the benefits of performance, flexibility, and integration make it a wise long-term investment.

1.5 Intrusion detection system

To support efficient threat detection within the SIEM solution, it is important to choose a reliable intrusion detection system.

1.5.1 Intrusion detection system definition

An intrusion detection system (IDS) is a network security tool that monitors network traffic and devices for known malicious activity, suspicious activity or security policy violations.

An IDS can help accelerate and automate network threat detection by alerting security administrators to known or potential threats, or by sending alerts to a centralized security tool. A centralized security tool such as a security information and event management (SIEM) system can combine data from other sources to help security teams identify and respond to cyberthreats that might slip by other security measures [28].

1.5.2 Functional architecture of an intrusion detection system

Intrusion Detection Systems rely on specific mechanisms to identify abnormal or unauthorized behavior within a network:

- An IDS (Intrusion Detection System) monitors the traffic on a computer network to detect any suspicious activity.
- It analyzes the data flowing through the network to look for patterns and signs of abnormal behavior.
- The IDS compares network activity to a set of predefined rules and patterns to identify any activity that might indicate an attack or intrusion.
- If the IDS detects something that matches one of these rules or patterns, it sends an alert to the system administrator.
- The system administrator can then investigate the alert and take action to prevent any damage or further intrusion [29].

Figure 1.3 presents the positioning of an IDS in relation to other network components [29]:

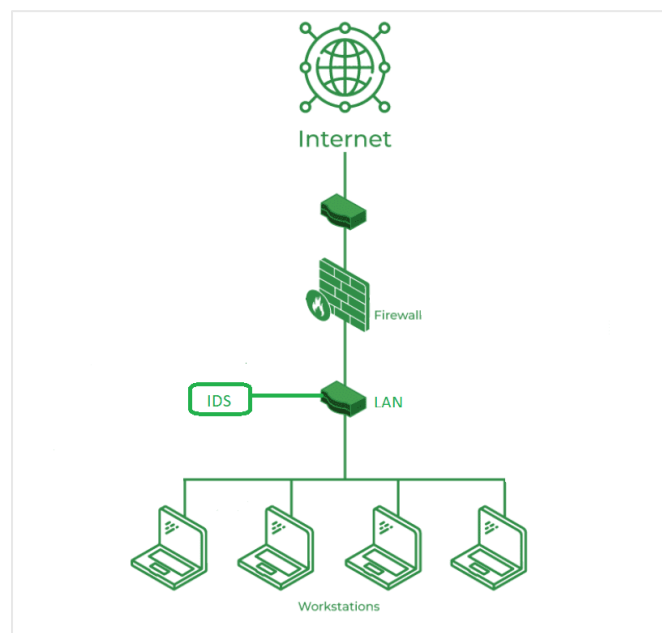


Figure 1.3: Integration of an IDS

1.5.3 Most popular intrusion detection systems tools

While most IDS tools share basic functionalities like threat detection and alerting, they differ significantly in scalability, protocol support, and integration options. The following table 1.2 summarizes these distinctions:

Table 1.2: Difference between the most popular IDS tools

Feature	Suricata	Snort	Zeek
Detection method	Signature, protocol/anomaly	Signature only	Behavior-based scripting
Performance	Multi-threaded, high-speed	Single-threaded	Depends on scripts
Protocol support	Deep, many protocols	Good	Extensive, scriptable
Rule compatibility	Supports Snort rules, own rules	Snort rules	Uses its own scripting
Logging	JSON/EVE, detailed	Traditional alerts	Detailed event logs
Use case	IDS/IPS, monitoring	IDS/IPS	Network behavior analysis

In summary, while Snort, Zeek, and Suricata each offer strong intrusion detection capabilities, Suricata proves to be the best choice for our SIEM project. It combines the signature-based detection of Snort with deeper protocol analysis and modern logging, while also offering high performance through multi-threading. Suricata's ability to generate detailed JSON logs makes it easy to integrate with our SIEM for real-time alerting and analysis. Its versatility as an IDS, IPS, and network monitoring tool ensures we have a complete and efficient security solution in our environment.

1.6 Log forwarding

To ensure that log data is efficiently collected and transmitted to the SIEM system, a reliable mechanism known as a log forwarder is used.

1.6.1 Log forwarding definition

Log forwarding refers to the process of gathering log data from a variety of sources, such as servers, applications, or devices, and sending them to some central location for storage, analysis, or monitoring. This central location might be a logging server, a Security Information and Event Management (SIEM) system, or a cloud-based log management service [30].

1.6.2 Log forwarders functions

In a SIEM pipeline, log forwarders serve as the initial stage of data ingestion, ensuring that security-relevant logs are reliably captured and delivered in real time. They often support different

log formats (e.g., Syslog, JSON) and transport protocols (e.g., TCP, UDP), and may apply basic filtering, parsing, or tagging before forwarding the data.

Their key functions include:

- Monitoring specific files or system events for log generation.
- Collecting logs from local or remote sources.
- Forwarding logs to the SIEM system for further analysis and correlation.
- Ensuring log integrity and timeliness in data transmission.

1.6.3 Most common log forwarders

While most log forwarders share basic functions like collecting and sending logs, they differ significantly in resource usage, parsing capabilities, and ease of integration.

The following table 1.3 summarizes these differences:

Table 1.3: difference between the most common log forwarders

Feature	Syslog	Logstash	Splunk Universal Forwarder
Purpose	Basic log forwarding	Log processing & forwarding	Forward logs to Splunk
Parsing	Minimal	Powerful & flexible	Minimal
Resource use	Very low	Medium	Low
Setup ease	Easy	Moderate	Easy
Integration	Works with many systems	Works well with ELK stack	Native Splunk integration
Platform	Unix/Linux	Cross-platform	Cross-platform

In conclusion, while Logstash and Splunk Universal Forwarder offer powerful log processing and native integration capabilities, Syslog stands out for its simplicity, lightweight nature, and broad compatibility. Logstash provides advanced parsing and transformation but requires more resources and complex setup, which can be unnecessary if our SIEM platform already handles log analysis and indexing. Splunk UF integrates directly with Splunk but still consumes more system resources and involves more maintenance compared to Syslog. For our SIEM project, where the focus is on efficient log forwarding from multiple Ubuntu clients to Splunk for centralized processing, Syslog is the best choice. Its minimal resource use, ease of configuration, and universal support ensure reliable and low-overhead log forwarding, keeping the client machines optimized and simplifying overall system management.

1.7 Conclusion

To sum up, this chapter introduced the context of the project and outlined the cybersecurity challenges faced by the organization. It presented SIEM as an effective solution, detailing its architecture, functions, and benefits. A comparison of SIEM tools justified the choice of Splunk, while the sections on intrusion detection systems and log forwarders explained the role and selection of tools like Suricata and Syslog. This theoretical foundation prepares the ground for the implementation phase discussed in the next chapter.

Chapter 2

Implementation of the SIEM solution

2.1 Introduction

This chapter presents the implementation of the solution, starting with a brief description of the global architecture and the role of every component moving to their installation and configuration.

2.2 Global architecture of the SIEM solution

This SIEM architecture is intended for scalability, redundancy, and efficient threat detection. Suricata, a high-performance, open-source intrusion detection and prevention system (IDS/IPS), is used to monitor networks. Suricata analyzes network traffic in real time and sends comprehensive alerts to Splunk via syslog. Instead of sending logs to a central Splunk instance, the client sends them directly to Splunk Indexer 1, which serves as the primary input and processing node.

The Splunk deployment is organized as an indexer cluster, with one cluster master node and three indexer peers. The cluster master oversees the synchronization of index replication, node status, and data integrity across the cluster. When Indexer 1 receives logs from the client, it indexes the data locally and performs any pre-configured correlation searches, alert conditions, or dashboard updates. It then copies the indexed data to Indexers 2 and 3, guaranteeing redundancy and high availability in accordance with Splunk's data replication requirements.

This setup ensures that even if Indexer 1 fails, the data remains accessible from the other indexers. Although only Indexer 1 is actively used for search and visualization in this setup, the replication across peers ensures future scalability where additional search heads or search head clustering can be introduced.

This architecture supports real-time threat detection, log integrity, and centralized monitoring. The following figure 2.1 gives a clear view of the solution's structure:

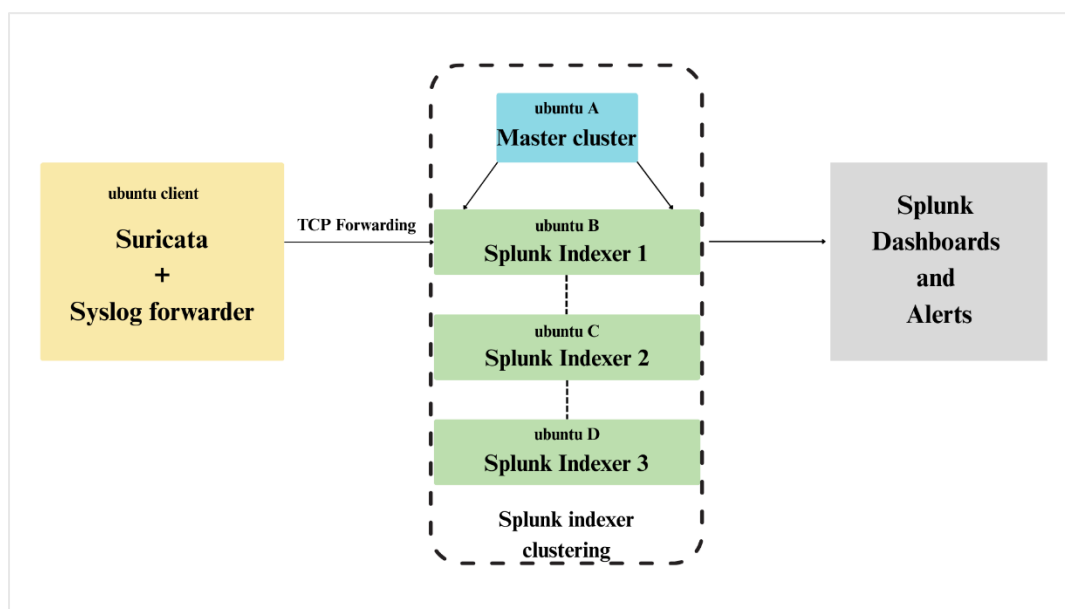


Figure 2.1: Architecture of the SIEM solution

2.2.1 Suricata

Suricata is a high performance, open-source network analysis and threat detection software used by most private and public organizations and embedded by major vendors to protect their assets [10]. The figure 2.2 shows Suricata logo [10]:



Figure 2.2: Suricata logo

There are 5 main modes of operation and data generation that Suricata does or is used for:

- IDS – Intrusion Detection System (default)

In this mode of operation, Suricata generates security events (alerts) based on the traffic inspection by signatures (rules), IoC matching (ex: hashes/domains/URLs/TLS/SSH fingerprints etc.), and/or Lua scripting. This mode is the least verbose in terms of log volume generation.

- IPS – Intrusion Prevention System

This mode of operation is very similar to the above in terms of log volume and traffic inspection. The major difference is that Suricata actively stops (drops) or allows traffic based on the network inspection outcome. In other words, it is not a passive sniffer but rather it is deployed inline.

- NSM – Network Security Monitoring System (default)

In this mode of operation, Suricata generates logs for any and all protocols, file transactions, file extraction, anomaly and flow logs without any matching or inspection. It does not need signatures to operate. This mode is the most verbose in terms of log volume.

- FPC – Full packet Capture

In this mode of operation, Suricata generates unconditional full packet capture of the traffic it inspects. This mode naturally requires a lot of storage.

- Conditional PCAP capture

This mode of operation is a subset of the FPC described above that generates deduplicated full session/flow PCAP capture for any alert. In other words: all alerts are accompanied by a full session PCAP of the flow that triggered the alert.

- FW – Firewall

In this mode of operation, Suricata is used as a firewall. There are some community members that actively use Suricata as a firewall in the cloud, for example, AWS [11].

In our project, we need Suricata to detect intrusions, create logs and warnings, and then send them to Syslog. For these reasons, IDS mode has been confirmed as the most appropriate mode of operation, since it is the default and most widely used mode for detection-focused environments. It passively analyzes traffic without interfering or blocking. Additionally, it generates security alerts based on rules and indicators of compromise (IoCs) and forwards them to Syslog.

2.2.2 Syslog

Syslog stands for System Logging Protocol and is a standard protocol used to send system log or event messages to a specific server, called a syslog server. It is primarily used to collect various device logs from several different machines in a central location for monitoring and review [11].

The following figure 2.3 illustrates syslog deployment [11]:

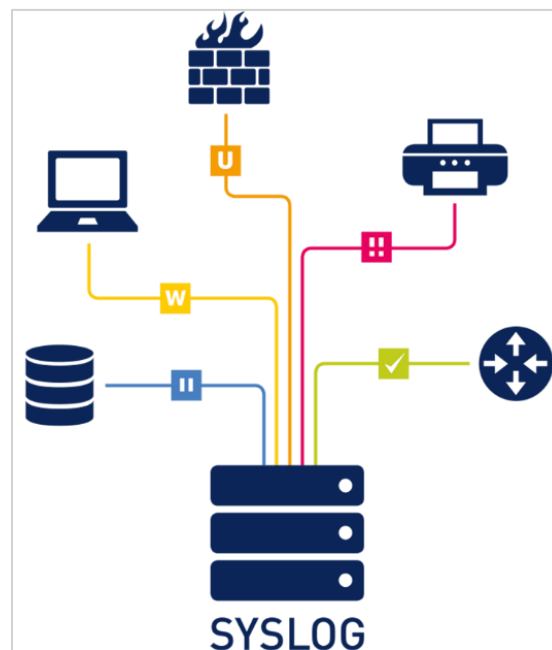


Figure 2.3: Syslog

Syslog is a protocol that consists of two components, a client and a server. The client sends log data across the network on port 514, while servers gather it and build centralized logs.

2.2.3 Splunk

Splunk is a technology that is used for searching, monitoring, visualizing, and analyzing machine data on a real-time basis. It is a tool for log management and analysis [12].

The following figure 2.4 shows Splunk logo [13]:



Figure 2.4: Splunk logo

2.2.3.1 Splunk architecture

Splunk is commonly used as a SIEM for collecting, indexing, and analyzing machine-generated data in real time. It operates by collecting data from a variety of sources, including servers, network devices, and applications, which is then indexed by Splunk Indexers. The indexed data is saved and made searchable. When a user initiates a search or query, the Search Head distributes the request to numerous Indexers, who process the data immediately and provide the results. This distributed search method enables Splunk to quickly manage huge amounts of data and provide quick insights via dashboards, alerts, and reports.

This diagram in figure 2.5 illustrates how Splunk operates [14]:

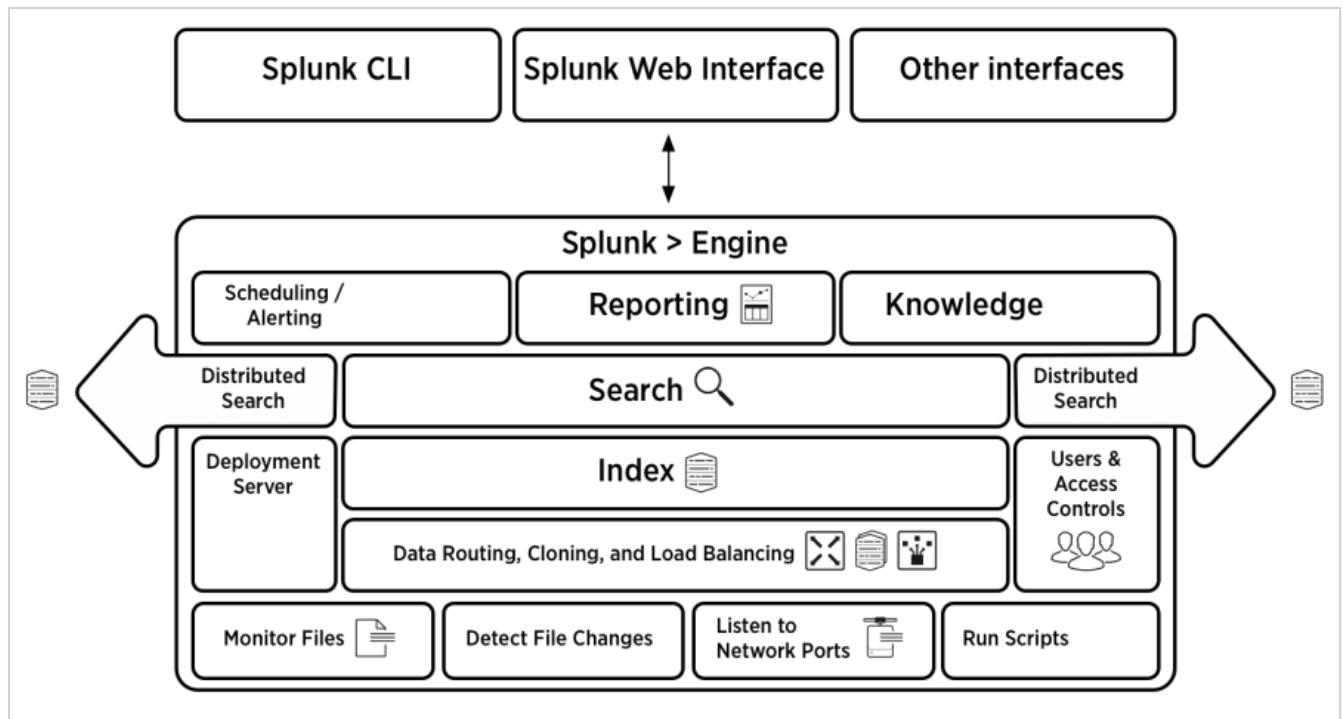


Figure 2.5: Splunk architecture

2.2.3.2 Splunk indexer clustering

Indexer clusters are groups of Splunk Enterprise indexers configured to replicate each other's data, so that the system keeps multiple copies of all data. This process is known as index replication. By maintaining multiple, identical copies of Splunk Enterprise data, clusters prevent data loss while promoting data availability for searching.

Indexer clusters feature automatic failover from one indexer to the next. This means that, if one or more indexers fail, incoming data continues to get indexed and indexed data continues to be searchable [26].

2.2.3.2.1 Indexer replication benefits

- **Data availability.** An indexer is always available to handle incoming data, and the indexed data is available for searching.
- **Data fidelity.** You never lose any data. You have assurance that the data sent to the cluster is exactly the same data that gets stored in the cluster and that a search can later access.
- **Data recovery.** Your system can tolerate downed indexers without losing data or losing access to data.
- **Disaster recovery.** With multisite clustering, your system can tolerate the failure of an entire data center.
- **Search affinity.** With multisite clustering, search heads can access the entire set of data through their local sites, greatly reducing long-distance network traffic [26].

2.2.3.2.2 Indexer cluster architecture

An indexer cluster is a group of Splunk Enterprise instances, or nodes, that, working in concert, provide a redundant indexing and searching capability. Each cluster has three types of nodes:

- A single manager node to manage the cluster.
- Several to many peer nodes to index and maintain multiple copies of the data and to search the data.
- One or more search heads to coordinate searches across the set of peer nodes.

The **manager node** manages the cluster. It coordinates the replicating activities of the peer nodes and tells the search head where to find data. It also helps manage the configuration of peer nodes and orchestrates remedial activities if a peer goes down.

The **peer nodes** receive and index incoming data, just like non-clustered, stand-alone indexers. Unlike stand-alone indexers, however, peer nodes also replicate data from other nodes in the cluster. A peer node can index its own incoming data while simultaneously storing copies of data from other nodes. You must have at least as many peer nodes as the replication factor. That is, to support a replication factor of 3, you need a minimum of three peer nodes [26].

The **search head** runs searches across the set of peer nodes. You must use a search head to manage searches across indexer clusters [26].

Here is a diagram in figure 2.6 of a basic, single-site indexer cluster, containing three peer nodes and supporting a replication factor of 3 [26]:

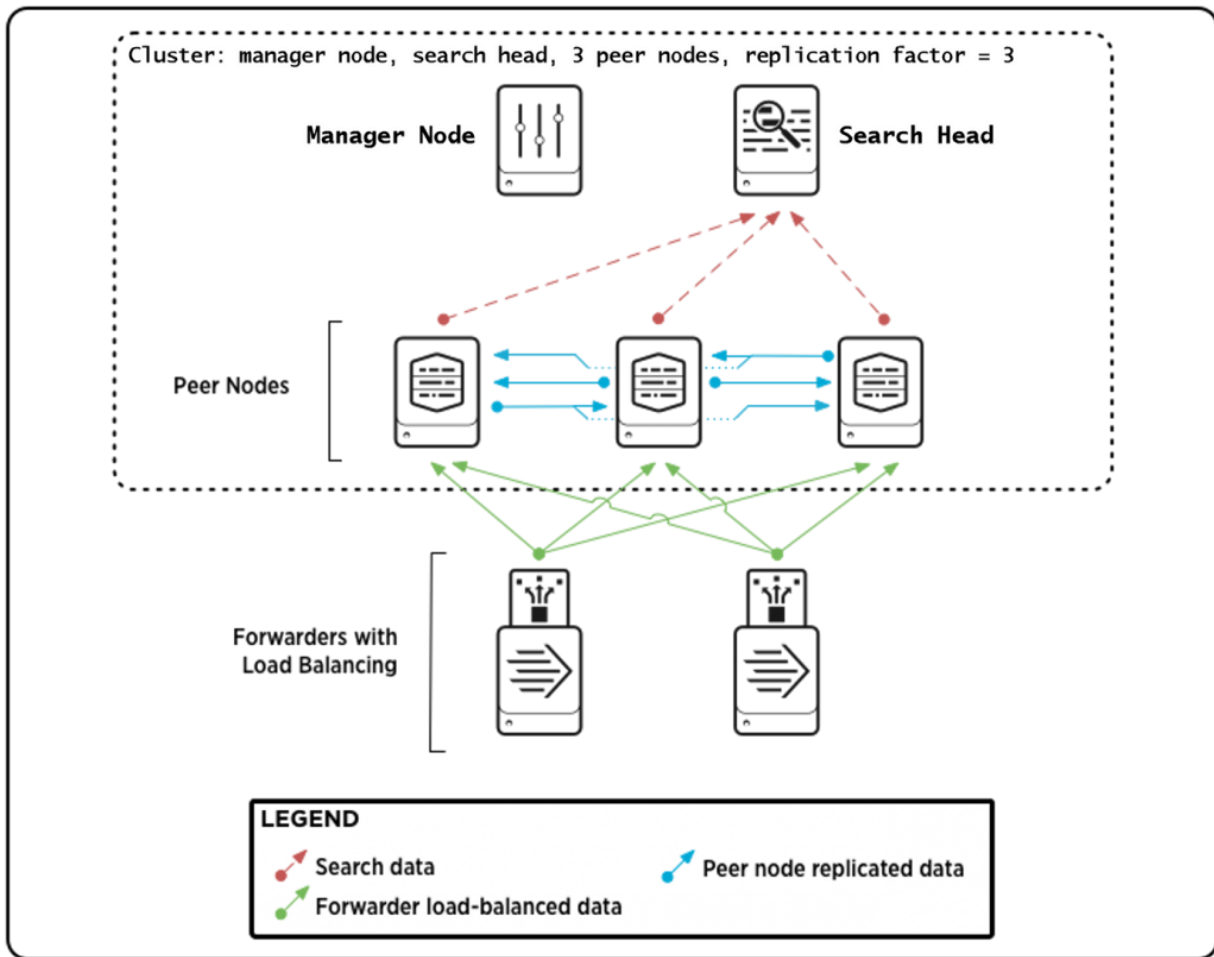


Figure 2.6: Splunk indexer cluster architecture

The illustration shows a distributed system that includes a manager node (which manages the cluster), a search head (which answers queries), and three peer nodes that store identical copies of data for redundancy.

Data is duplicated across all peers to ensure fault tolerance. The search head distributes requests, and load balancing improves performance.

2.3 System setup tools

2.3.1 VMware workstation

VMware Workstation is the first product ever released by the software company. It enables users to create and run VMs directly on a single Windows or Linux desktop or laptop. Those VMs run simultaneously with the physical machine. Each VM runs its own OS such as Windows or Linux [15].

Behold, in figure 2.4, the logo of VMware workstation [16]:



Figure 2.7: VMware workstation logo

2.3.2 Ubuntu virtual machine

Ubuntu is a Debian-based Linux operating system that runs from the desktop to the cloud, to all your internet connected things. It is the world's most popular operating system across public clouds and OpenStack clouds [17].

The next figure 2.8 shows ubuntu logo [18]:



Figure 2.8: Ubuntu logo

The ubuntu virtual machine is chosen for its stability, integration with open-source tools and capability to operate all Suricata, Syslog and Splunk effectively. This machine functions as both a sensor and a log collector/processor at the same time.

2.4 Implementation

After setting up the virtual environment using VMware and installing Ubuntu as the base operating system for all the virtual machines, the following step is to install and configure the essential components.

2.4.1 Installation and configuration of Suricata

2.4.1.1 Installation

To install Suricata on ubuntu client VM, we utilized the “apt” package manager to get and install it straight from the official repository using the command “sudo apt install suricata -y” as shown in figure 2.9. Suricata was now ready to monitor network traffic and issue alarms.

The configuration files are placed in “/etc/suricata/”, while the primary log output is saved in “/var/log/suricata/eve.json”.

```
rabeb@rabeb-virtual-machine:~$ sudo apt install suricata -y
```

Figure 2.9: Suricata installation

We started and enabled Suricata then, we made sure that it was running by executing these commands in figure 2.10:

```
root@rabeb-virtual-machine:~# sudo systemctl start suricata
root@rabeb-virtual-machine:~# sudo systemctl status suricata
● suricata.service - LSB: Next Generation IDS/IPS
   Loaded: loaded (/etc/init.d/suricata; generated)
   Active: active (exited) since Mon 2025-04-14 12:08:34 CET; 25min ago
     Docs: man:systemd-sysv-generator(8)
    CPU: 157ms

12:08:34 14 أبريل 2025 rabeb-virtual-machine systemd[1]: Starting LSB: Next Generation IDS/IPS...
12:08:34 14 أبريل 2025 rabeb-virtual-machine suricata[44271]: Starting suricata in IDS (af-packet) m>
12:08:34 14 أبريل 2025 rabeb-virtual-machine systemd[1]: Started LSB: Next Generation IDS/IPS.
lines 1-9/9 (END)
```

Figure 2.10: Suricata status

2.4.1.2 Configuration

For the configuration phase, we edited “/etc/suricata/suricata.yaml” file to change the default network interface to the VM’s network interface as illustrated in figure 2.11:

```
af-packet:
  - interface: ens33
```

Figure 2.11: Network interface configuration

2.4.2 Installation and configuration of Syslog

2.4.2.1 Installation

To install Syslog on the same ubuntu client VM, we used the default package available in Ubuntu by running the following command in figure 2.12:

```
root@rabeb-virtual-machine:~# sudo apt install rsyslog
```

Figure 2.12: Syslog installation

We started and enabled syslog then, we made sure that it is running by executing these commands in figure 2.13:

```

root@rabeb-virtual-machine:~# sudo systemctl start rsyslog
root@rabeb-virtual-machine:~# sudo systemctl enable rsyslog
root@rabeb-virtual-machine:~# sudo systemctl status rsyslog
● rsyslog.service - System Logging Service
   Loaded: loaded (/lib/systemd/system/rsyslog.service; enabled; vendor preset: enabled)
   Active: active (running) since Mon 2025-04-14 11:37:12 CET; 1h 48min ago
 TriggeredBy: ● syslog.socket
    Docs: man:rsyslogd(8)
          man:rsyslog.conf(5)
          https://www.rsyslog.com/doc/
   Main PID: 787 (rsyslogd)
     Tasks: 4 (limit: 9383)
    Memory: 3.1M
       CPU: 748ms
    CGroup: /system.slice/rsyslog.service
            └─787 /usr/sbin/rsyslogd -n -iNONE

11:37:11 14 أفري rabeb-virtual-machine systemd[1]: Starting System Logging Service...
11:37:12 14 أفري rabeb-virtual-machine rsyslogd[787]: imuxsock: Acquired UNIX socket '/run/sys
11:37:12 14 أفري rabeb-virtual-machine rsyslogd[787]: rsyslogd's groupid changed to 111
11:37:12 14 أفري rabeb-virtual-machine systemd[1]: Started System Logging Service.
11:37:12 14 أفري rabeb-virtual-machine rsyslogd[787]: rsyslogd's userid changed to 104
11:37:12 14 أفري rabeb-virtual-machine rsyslogd[787]: [origin software="rsyslogd" swVersion="8
lines 1-20/20 (END)

```

Figure 2.13: Syslog status

2.4.2.2 Configuration

To make syslog forward all system logs to Splunk, we added a forwarding rule in syslog configuration. Logs will be forwarded to a remote syslog receiver which is Splunk, using Splunk server IP on port 514 as seen in figure 2.14:

```

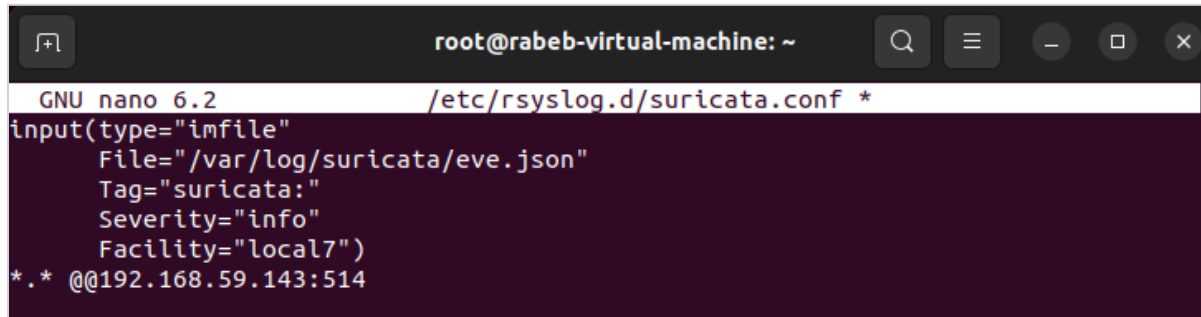
root@rabeb-virtual-machine: ~
GNU nano 6.2 /etc/rsyslog.d/50-default.conf *
#      cron,daemon.none;\
#      mail,news.none      -/var/log/messages
#
# Emergencies are sent to everybody logged in.
#
*.emerg                          :omusrmsg:*
#
# I like to have messages displayed on the console, but only on a virtual
# console I usually leave idle.
#
#daemon,mail.*;\
#      news.=crit;news.=err;news.=notice;\
#      *.=debug;*.=info;\
#      *.=notice;*.=warn      /dev/tty8
*. * @192.168.59.143:514

```

Figure 2.14: syslog configuration for system logs

The next step is to configure Syslog to forward Suricata logs to Splunk. We set up a custom configuration file at “/etc/rsyslog.d/suricata.conf”. In this file, we utilized the “imfile” module to

monitor Suricata's primary JSON log file, which is stored at “/var/log/suricata/eve.json”. We added the tag “suricata:” to assist identifying the logs and set the destination to 192.168.59.143:514, Splunk's port, for receiving logs, as illustrated in figure 2.15. This configuration guarantees that only Suricata logs are transmitted to Splunk, with system logs handled separately.



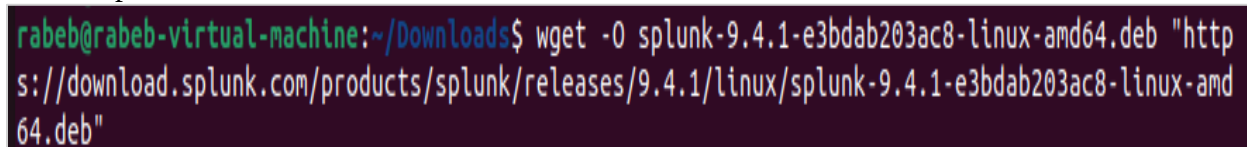
```
root@rabeb-virtual-machine: ~  
GNU nano 6.2 /etc/rsyslog.d/suricata.conf *  
input(type="imfile"  
      File="/var/log/suricata/eve.json"  
      Tag="suricata:"  
      Severity="info"  
      Facility="local7")  
*. * @@192.168.59.143:514
```

Figure 2.15: Syslog configuration for Suricata logs

2.4.3 Installation and configuration of Splunk

2.4.3.1 Installation

To install Splunk for an indexer clustering, we first downloaded the “.deb” package from the official website and then installed it with “dpkg”, as illustrated in figure 2.16, on the four ubuntu VMs: one to serve as the Cluster Manager and three as Indexer Peers. After installation, we launched Splunk and approved the licensing agreement. We then set Splunk to run immediately on startup.



```
rabeb@rabeb-virtual-machine:~/Downloads$ wget -O splunk-9.4.1-e3bdab203ac8-linux-amd64.deb "http  
s://download.splunk.com/products/splunk/releases/9.4.1/linux/splunk-9.4.1-e3bdab203ac8-linux-am  
d64.deb"
```

Figure 2.16: Splunk installation

2.4.3.2 Configuration

After installing Splunk in each ubuntu VM, we used <http://localhost:8000> to access the web interface and configure it and we set the admin credentials at the initial login.

2.4.3.2.1 Configuration of the initial Splunk instance

The deployment started with a single Splunk instance deployed on an Ubuntu VM, which served as the central log receiver. This instance was set up to accept log data from a different Ubuntu client via syslog on TCP port 514. We defined the source type as “syslog”, assigned the proper index, and confirmed the input as displayed in figure 2.17. This configuration enabled Splunk to accept index logs delivered by syslog, including Suricata alerts. It performed the core Splunk functions of data processing, indexing, and searching, providing real-time visibility into network activities.

Review

Input Type TCP Port
 Port Number 514
 Source name override syslog
 Restrict to Host N/A
 Source Type suricata:json
 App Context search
 Host (IP address of the remote server)
 Index main

Figure 2.17: Splunk data input configuration

To improve threat detection, we set up alerts in Splunk to monitor security events generated by Suricata. We used the Search and Reporting tool to generate personalized search queries that targeted certain incursion patterns. These inquiries were then recorded as alerts with specific trigger factors such as the number of occurrences within a given time range. Alert actions were set up to inform administrators via trigger automated responses and send alerts on Splunk mobile App. This structure assures fast detection of suspicious behavior and enhances our capacity to respond promptly to possible threats.

The following figure 2.18 shows alert settings:

Edit Alert

Settings

Alert: splunk alerts

Description: Optional

Alert type: Scheduled | Real-time

Expires: 24 day(s)

Trigger Conditions

Trigger alert when: Per-Result

Throttle: ☐

Trigger Actions

+ Add Actions

When triggered: ▼ Add to Triggered Alerts Remove

Severity: High

Figure 2.18: Splunk alerts

After setting up the alerts, the figure 2.19 shows how the triggered alerts field would look like:

	Time	Alert name	App	Type	Severity	Mode	Actions
☐	2025-04-22 17:22:01 CET	splunk alerts	search	Real-time	High	Per Result	View Results Edit Search Delete
☐	2025-04-22 17:22:01 CET	splunk alerts	search	Real-time	High	Per Result	View Results Edit Search Delete
☐	2025-04-22 17:22:01 CET	splunk alerts	search	Real-time	High	Per Result	View Results Edit Search Delete
☐	2025-04-22 17:15:10 CET	splunk alerts	search	Real-time	High	Per Result	View Results Edit Search Delete
☐	2025-04-22 17:15:08 CET	splunk alerts	search	Real-time	High	Per Result	View Results Edit Search Delete
☐	2025-04-22 17:12:45 CET	splunk alerts	search	Real-time	High	Per Result	View Results Edit Search Delete
☐	2025-04-22 17:12:45 CET	splunk alerts	search	Real-time	High	Per Result	View Results Edit Search Delete
☐	2025-04-22 17:11:11 CET	splunk alerts	search	Real-time	High	Per Result	View Results Edit Search Delete
☐	2025-04-22 17:11:10 CET	splunk alerts	search	Real-time	High	Per Result	View Results Edit Search Delete
☐	2025-04-18 21:43:59 CET	splunk alerts	search	Real-time	High	Per Result	View Results Edit Search Delete
☐	2025-04-18 21:43:59 CET	splunk alerts	search	Real-time	High	Per Result	View Results Edit Search Delete
☐	2025-04-18 21:43:43 CET	splunk alerts	search	Real-time	High	Per Result	View Results Edit Search Delete
☐	2025-04-18 21:43:42 CET	splunk alerts	search	Real-time	High	Per Result	View Results Edit Search Delete
☐	2025-04-18 21:43:37 CET	splunk alerts	search	Real-time	High	Per Result	View Results Edit Search Delete
☐	2025-04-18 21:43:37 CET	splunk alerts	search	Real-time	High	Per Result	View Results Edit Search Delete
☐	2025-04-18 21:43:27 CET	splunk alerts	search	Real-time	High	Per Result	View Results Edit Search Delete
☐	2025-04-18 21:43:27 CET	splunk alerts	search	Real-time	High	Per Result	View Results Edit Search Delete

Figure 2.19: Triggered alerts

To visualize system and network activities, we created Splunk dashboards for logs received from Syslog. Using the Search & Reporting app, we designed panels that provide real-time and historical data, including system events, user logins, service activities and Suricata logs. The figure 2.20 presents a dashboard data table of syslog. It provides an organized, real-time view of system log data gathered from ubuntu client using the syslog protocol. It usually comprises fields like timestamp, host, source, sourcetype, index, and raw message content.

_raw	_serial	_si	_sourcetype	_time	host	index	linecount	source
<190>Apr 15 22:07:45 rabebe-virtual-machine suricata:	0	rabebe-virtual-machine main	_json	2025-04-15T22:07:45.000+01:00	127.0.0.1	main	1	syslog

Figure 2.20: Splunk table dashboard

We use this table to monitor system activity across several servers or network devices by presenting suricata events, logs for login attempts, sudo commands, service status changes, and other vital system events. It may filter and search the data by time, host, severity, or keywords, making it an

effective tool for system monitoring, troubleshooting, and security analysis in a centralized Splunk environment.

In addition, we created a second dashboard focusing on data volume, which shows the amount of syslog entries over time as indicated in figure 2.21. This allows us to monitor log ingestion rates, discover anomalies or exceptions in event creation, and ensure that our logging system is working properly.

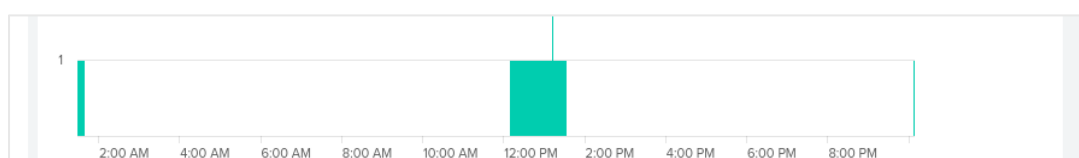


Figure 2.21: Data volume dashboard

2.4.3.2.2 Configuration of Splunk indexer clustering

To improve scalability, fault tolerance, and high availability, the standalone instance was combined with a Splunk indexer clustering architecture.

A dedicated Ubuntu VM was configured as the Cluster Manager, responsible for managing the cluster's replication and peer coordination. After installing Splunk Enterprise on it, the instance is configured in manager mode by enabling indexer clustering in Splunk through the web interface, specifically for setting up the Cluster Manager or the Master node as indicated in figure 2.22:

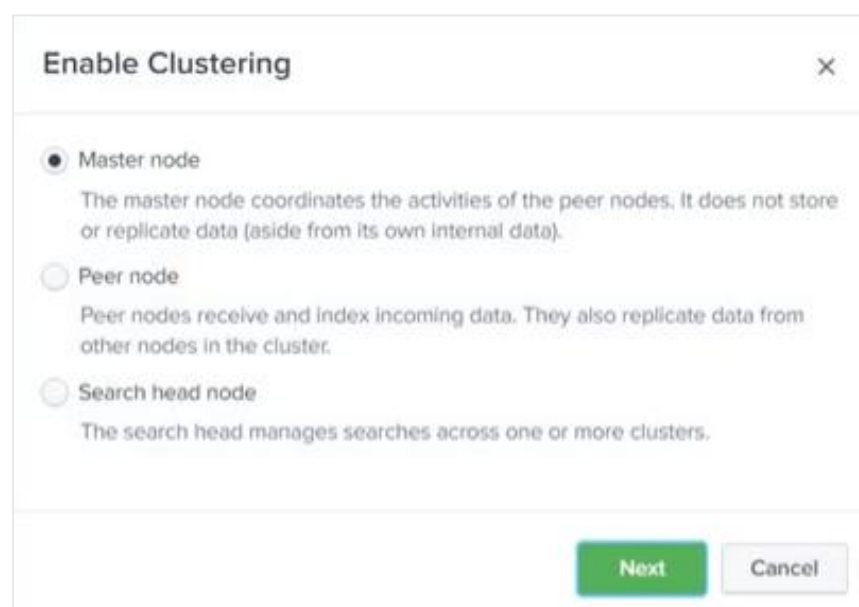
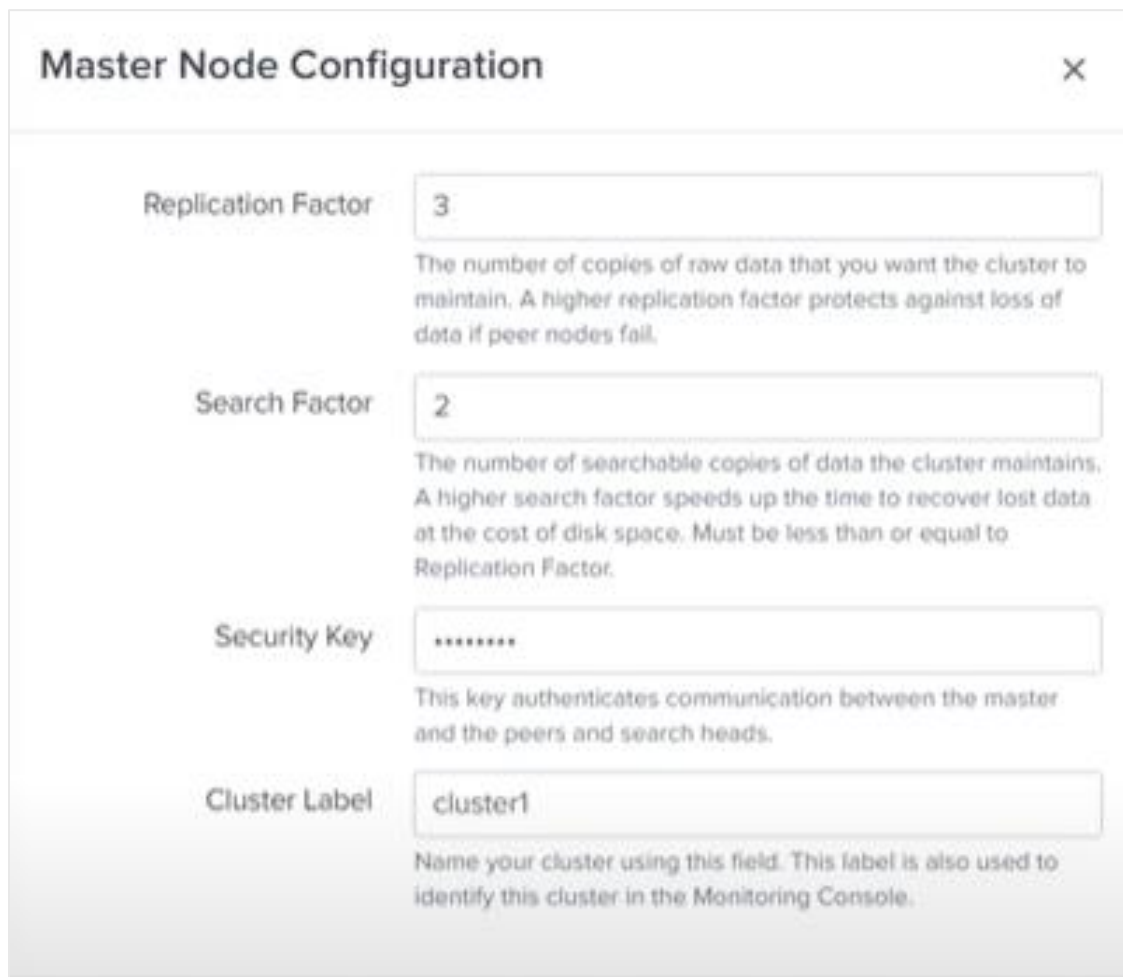


Figure 2.22: Enable clustering

The Master Node coordinates the operations of the peer nodes but does not store or index data. The setup specifies a replication factor of 3, which guarantees that three copies of each piece of raw data are dispersed across the cluster, resulting in high availability and resilience in the event of a node loss. A search factor of 2 is also specified, implying that at least two of the duplicated

copies are searchable, which improves search performance and reliability. A security key is supplied to securely authenticate communication between the manager and peer nodes, allowing only authorized members to join the cluster.

The figure 2.23 illustrates the master node configuration:

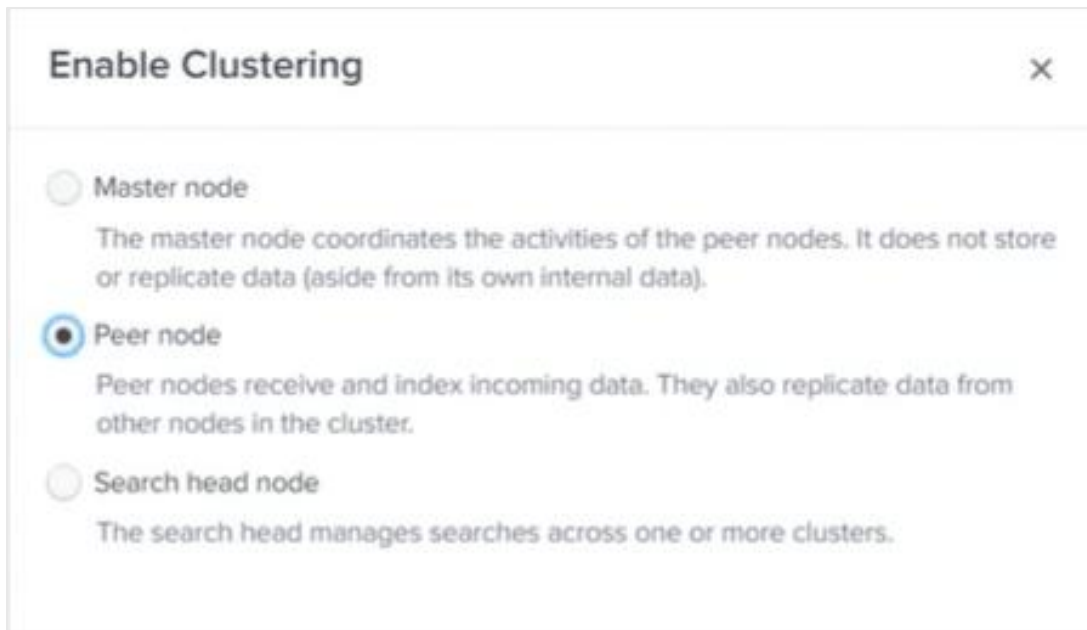


The image shows a 'Master Node Configuration' dialog box with a close button (X) in the top right corner. It contains four configuration fields, each with a label, a text input box, and a descriptive paragraph below it.

Field Label	Value	Description
Replication Factor	3	The number of copies of raw data that you want the cluster to maintain. A higher replication factor protects against loss of data if peer nodes fail.
Search Factor	2	The number of searchable copies of data the cluster maintains. A higher search factor speeds up the time to recover lost data at the cost of disk space. Must be less than or equal to Replication Factor.
Security Key	*****	This key authenticates communication between the master and the peers and search heads.
Cluster Label	cluster1	Name your cluster using this field. This label is also used to identify this cluster in the Monitoring Console.

Figure 2.23: Master node configuration

The next step is to enable the Peer Node role on all three Splunk Enterprise instances, including the original standalone instance that receives data from the client as outlined in figure 2.24. This instance will now function as Indexer 1 and replicate its data to the two additional indexers within the cluster.

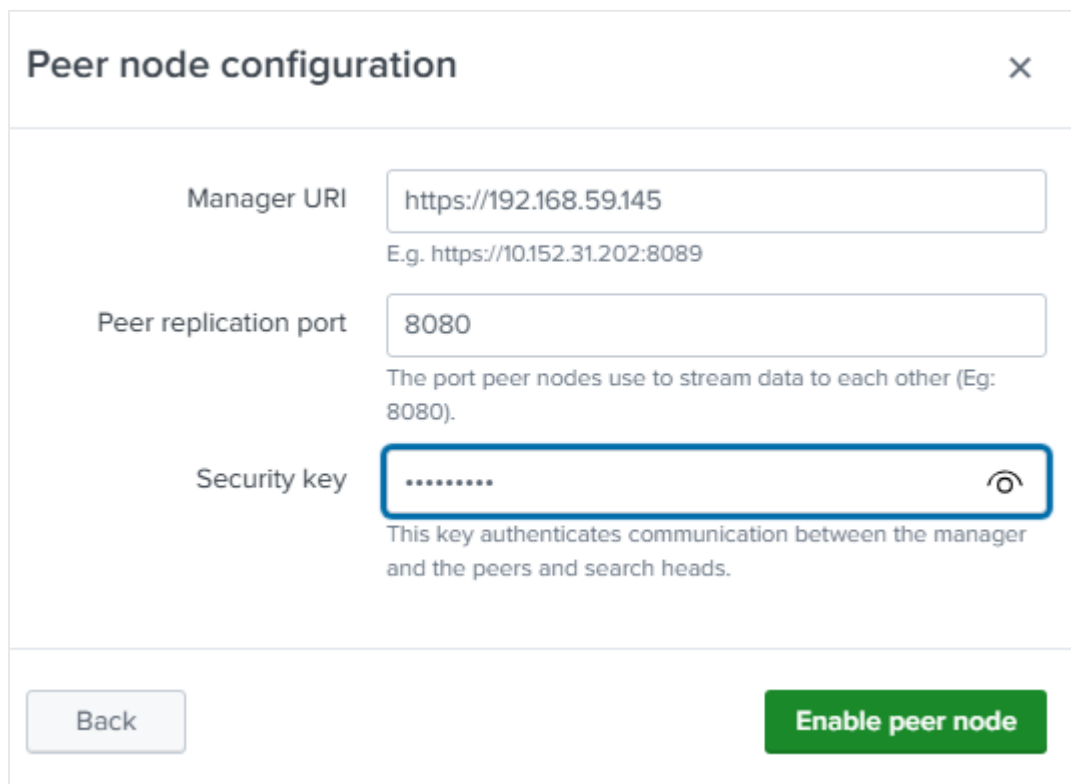


The dialog box titled "Enable Clustering" has a close button (X) in the top right corner. It contains three radio button options, each with a description:

- ☐ Master node
The master node coordinates the activities of the peer nodes. It does not store or replicate data (aside from its own internal data).
- ☒ Peer node
Peer nodes receive and index incoming data. They also replicate data from other nodes in the cluster.
- ☐ Search head node
The search head manages searches across one or more clusters.

Figure 2.24: Enable Peer node

Once "Peer node" is selected, the next step involves providing essential clustering parameters such as the manager node's address, Peer replication port and the security key to join the correct cluster as indicated in figure 2.25:



The dialog box titled "Peer node configuration" has a close button (X) in the top right corner. It contains three input fields with labels and descriptions:

- Manager URI**:
E.g. https://10.152.31.202:8089
- Peer replication port**:
The port peer nodes use to stream data to each other (Eg: 8080).
- Security key**: 
This key authenticates communication between the manager and the peers and search heads.

At the bottom, there are two buttons: "Back" and "Enable peer node".

Figure 2.25: Peer node configuration

This guarantees that the peer interacts securely with the cluster manager, joins the appropriate indexer cluster group, and contributes to data replication and search availability. These procedures are performed for each peer node in the cluster, including the ingesting indexer (Indexer 1) and the replicating peers (Indexers 2 and 3), resulting in a unified and fault-tolerant Splunk indexing cluster.

Q

10 per page ▼

i	Peer Name ↕	Fully Searchable ↕	Status ↕
>	indexer-3	✓ Yes	Up
>	indexer-1	✓ Yes	Up
>	indexer-2	✓ Yes	Up

Figure 2.26: Peer nodes

Finally, the master node displays three indexers (indexer-1, indexer-2, and indexer-3), each with Status Up and Fully Searchable, as displayed in figure 2.26, confirming that they are actively involved in the cluster.

With this configuration, indexer 1 automatically duplicates data to indexers 2 and 3, providing redundancy and fault tolerance.

2.5 Conclusion

To summarize, this chapter covered the architecture and setup of our lab environment, focusing on the implementation of essential security tools including Suricata for intrusion detection, Syslog for log forwarding, and Splunk for centralized log analysis. These tools were deployed on an Ubuntu client machine that continuously monitors and forwards logs to the Splunk environment. Initially, a single Splunk instance was used to receive, index, and analyze incoming logs, with dashboards and alerts configured for real-time visibility. To enhance scalability and ensure high availability, this instance was later integrated into a Splunk indexer clustering setup, consisting of a Cluster Manager and three peer nodes. The original standalone Splunk instance was reconfigured as Indexer 1 to ingest and replicate data to Indexer 2 and Indexer 3 under the coordination of the Cluster Manager. This clustered architecture provides redundancy, improved search performance, and fault tolerance, forming a robust and scalable foundation for our SIEM project, enabling efficient monitoring, threat detection, and response to potential security incidents.

Chapter 3

Test and validation of the SIEM solution

3.1 Introduction

In this chapter, we execute an attack simulation to evaluate the efficiency of our security monitoring system. We intend to verify the success of our solution by following a stated strategy. This test helps us to validate the entire system's integrity and performance in a real-world environment.

3.2 Attack strategy

The simulated attack provides a planned, multi-phase strategy to obtain unauthorized access to the victim machine. The first phase, Reconnaissance, includes identifying open ports and operating services. During the Gaining Access phase, we executed a brute-force attack against a target service using a known login and a customized password list. Once access is granted, Privilege Escalation is accomplished by confirming and exploiting existing administrator privileges. During the Persistence phase, the attacker installs a new privileged user to guarantee persistent system access. Finally, the Covering Tracks phase involves deleting security logs to remove evidence of the attack and lower the risk of discovery.

3.2.1 Attack tools

To realize this attack scenario, we used the following tools:

3.2.1.1 Kali Linux

Kali Linux (*formerly known as BackTrack Linux*) is an open-source, Debian-based Linux distribution which allows users to perform advanced penetration testing and security auditing. It runs on multiple platforms and is freely available and accessible to both information security professionals and hobbyists [19].

Behold the logo of Kali Linux in figure 3.1 [20]:



Figure 3.1: kali Linux logo

3.2.1.2 Nmap

Nmap is a network scanning tool—an open source Linux command-line tool—used for network exploration, host discovery, and security auditing. In other words, you can use Nmap to scan IP addresses, search for security loopholes, and scan for open ports on your computer network by sending packets and analyzing the responses [21].

The next figure 3.2 shows Nmap logo [22]:

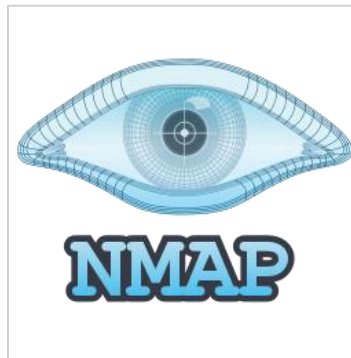


Figure 3.2: Nmap logo

3.2.1.3 Zenmap

Zenmap is the official Nmap Security Scanner GUI. It is a multi-platform, free open-source application that provides users with an easy-to-use graphical user interface (GUI) for Nmap [23].

3.2.1.4 Hydra

Hydra is a password cracking tool to perform brute force attacks against network services. The tool is used by professional ethical hackers to test the strength of passwords and assess the security posture of organizations.

However, attackers may also attempt to use it to perform malicious actions such as credential compromise and laterally move within the network [24].

The next figure 3.3 shows Hydra logo [25]:



Figure 3.3: Hydra logo

3.3 Attack execution

The attack scenario was executed in five phases:

3.3.1 Reconnaissance

The first phase includes identifying open ports and operating services. We executed a full scan using the graphical tool Zenmap as shown in figure 3.4:

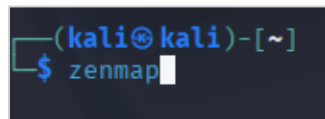


Figure 3.4: Zenmap

The scan discovered multiple open ports including SSH port 22, as observed in figure 3.5, which was very useful for exploitation.

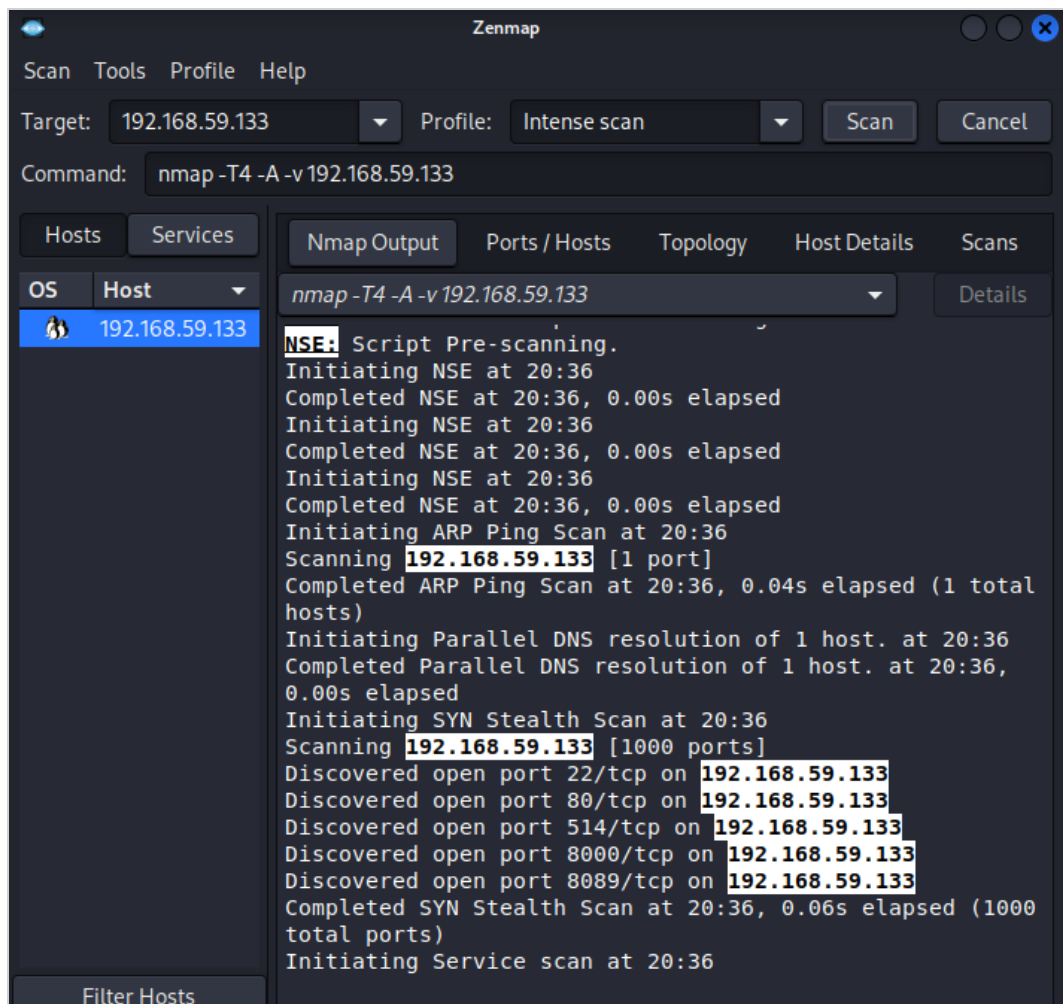
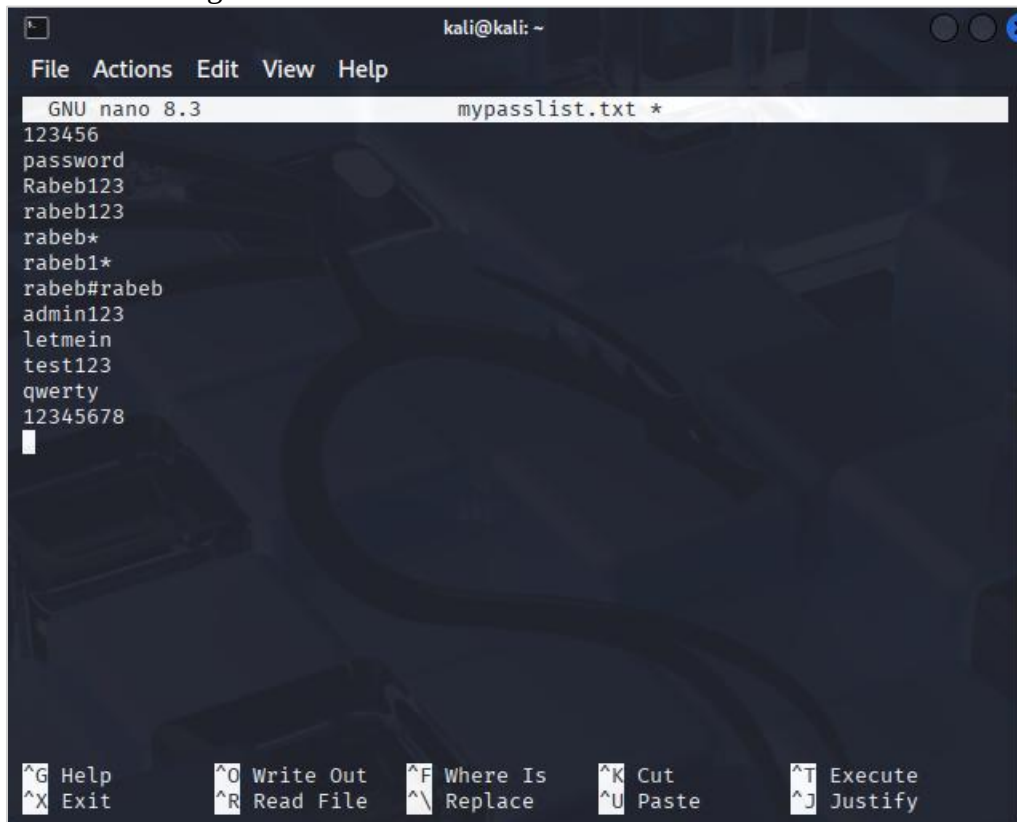


Figure 3.5: Port scan

3.3.2 Brute force

After checking that the SSH service was open on the target system, we launched a brute-force login attempt. In this case, we already had the right username. To improve efficiency and relevance, we built a personalized password list using context-specific guesses rather than a public wordlist as outlined in figure 3.6:



The image shows a terminal window titled 'kali@kali: ~' with a menu bar (File, Actions, Edit, View, Help). The editor is GNU nano 8.3, editing a file named 'mypasslist.txt'. The file contains the following text:

```
123456
password
Rabeb123
rabeb123
rabeb*
rabeb1*
rabeb#rabeb
admin123
letmein
test123
qwerty
12345678
```

At the bottom of the terminal, there is a status bar with keyboard shortcuts: ^G Help, ^X Exit, ^O Write Out, ^R Read File, ^F Where Is, ^\ Replace, ^K Cut, ^U Paste, ^T Execute, and ^J Justify.

Figure 3.6: A customized password list

After creating our customized password list, we executed our brute-force attack using Hydra tool to extract login credentials as shown in the next figure 3.7:

```

(kali@kali)-[~]
$ hydra -l rabe1 -P mypasslist.txt ssh://192.168.59.133

Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in
military or secret service organizations, or for illegal purposes (this is n
on-binding, these ** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2025-04-17 09:
30:24
[WARNING] Many SSH configurations limit the number of parallel tasks, it is r
ecommended to reduce the tasks: use -t 4
[WARNING] Restorefile (you have 10 seconds to abort... (use option -I to skip
waiting)) from a previous session found, to prevent overwriting, ./hydra.res
tore
[DATA] max 13 tasks per 1 server, overall 13 tasks, 13 login tries (l:1/p:13)
, ~1 try per task
[DATA] attacking ssh://192.168.59.133:22/
[22][ssh] host: 192.168.59.133 login: rabe1 password: rabe1*
1 of 1 target successfully completed, 1 valid password found
[WARNING] Writing restore file because 2 final worker threads did not complet
e until end.
[ERROR] 2 targets did not resolve or could not be connected
[ERROR] 0 target did not complete
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2025-04-17 09:
30:39

```

Figure 3.7: Brute-force attack

Once we got the password, we logged into the SSH service onto the target machine as shown in figure 3.8:

```

(kali@kali)-[~]
$ ssh rabe1@192.168.59.133

rabe1@192.168.59.133's password:
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 6.8.0-57-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

11 updates can be applied immediately.
10 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

New release '24.04.2 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

rabe1@rabe1-virtual-machine:~$

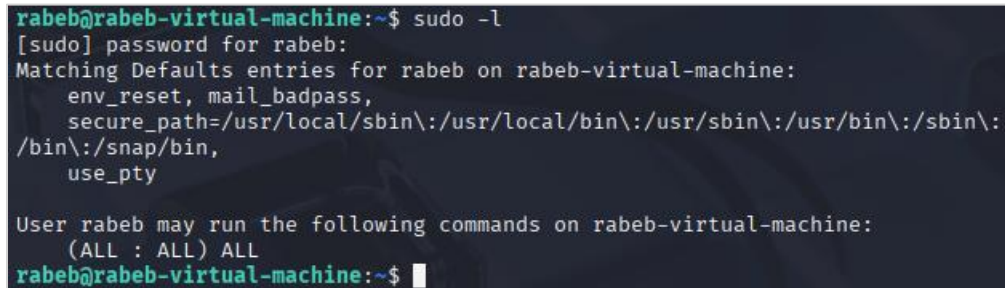
```

Figure 3.8: SSH login

3.3.3 Privilege Escalation

Once inside the system, we checked the current user's rights. In this instance, root access was already available via Sudo, so we checked the permissions.

The following figure 3.9 confirms that we can execute any command as the root user:



```
rabebe@rabebe-virtual-machine:~$ sudo -l
[sudo] password for rabebe:
Matching Defaults entries for rabebe on rabebe-virtual-machine:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin,
    use_pty

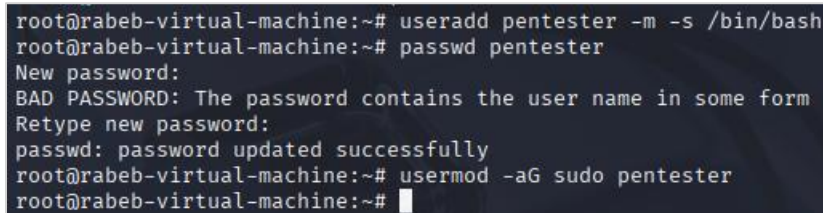
User rabebe may run the following commands on rabebe-virtual-machine:
    (ALL : ALL) ALL
rabebe@rabebe-virtual-machine:~$
```

Figure 3.9: Sudo privileges check

3.3.4 Persistence

To continue access to the hacked system, we created a new user account with administrator privileges. This assures that even if the initial access is lost or the password changed, we can still access the account.

We created a new user named “pentester” and added it to the Sudo group, granting administrative privileges as demonstrated in figure 3.10:

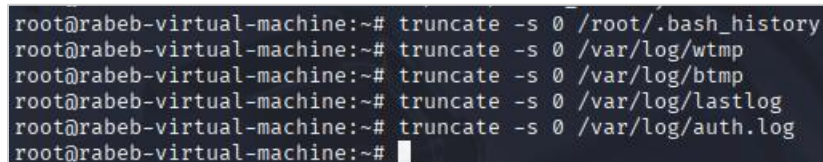


```
root@rabebe-virtual-machine:~# useradd pentester -m -s /bin/bash
root@rabebe-virtual-machine:~# passwd pentester
New password:
BAD PASSWORD: The password contains the user name in some form
Retype new password:
passwd: password updated successfully
root@rabebe-virtual-machine:~# usermod -aG sudo pentester
root@rabebe-virtual-machine:~#
```

Figure 3.10: Creation of a new user

3.3.5 Tracks cover

In the last step of the attack, we tried to remove all signs of our presence and actions from the hacked machine. This was accomplished by removing crucial log files containing system events, login attempts, shell history, and authentication logs as demonstrated in figure 3.11:



```
root@rabebe-virtual-machine:~# truncate -s 0 /root/.bash_history
root@rabebe-virtual-machine:~# truncate -s 0 /var/log/wtmp
root@rabebe-virtual-machine:~# truncate -s 0 /var/log/btmp
root@rabebe-virtual-machine:~# truncate -s 0 /var/log/lastlog
root@rabebe-virtual-machine:~# truncate -s 0 /var/log/auth.log
root@rabebe-virtual-machine:~#
```

Figure 3.11: Log files remove

3.4 Detection and alerting

Throughout the simulated attack, a network-based intrusion detection system continuously monitored traffic on the victim's machine. Suricata, which operated as the detection engine, collected and analyzed network events in real time.

Suricata was configured as part of the monitoring process to transmit alarms and log data to Splunk via syslog. This connection enabled the centralized visibility and correlation of security events.

After the attack occurred out, Splunk displayed many alarms under the Triggered alarms area. The notifications identified the several stages of the attack, including:

- connectivity test using “ping”, as illustrated in figure 3.12:

Time	Event
4/17/25 3:11:38.000 PM	<131>Apr 17 15:11:38 rabebe-virtual-machine suricata[1254]: [1:1000001:1] Ping detected [host=192.168.0.1 source=192.168.0.1 sourceip=192.168.0.1]

Figure 3.12: Ping detection

- Nmap scan, as illustrated in figure 3.13:

Time	Event
4/17/25 3:10:40.000 PM	<174>Apr 17 15:10:40 rabebe-virtual-machine suricata: 04/17/2025-15:10:40.494004 [**] [1:1000007:1] Nmap SYN Scan detected [host=192.168.0.1 source=192.168.0.1 sourceip=192.168.0.1]

Figure 3.13: Nmap detection

- Brute-force attack using Hydra, as illustrated in figure 3.14:

```
[ALERT] SSH Brute Force Detected - Possible Hydra [**] [Classification: (null)] [Priority: 3] {TCP} 192.168.59.130:60118 -> 192.168.59.13
```

Figure 3.14: Brute-force detection

- Creation of a new user named “pentester”, as illustrated in figure 3.15:

```
rabebe-virtual-machine useradd[8005]: new user: name=pentester, UID=1002, GID=1002, home=/home/pentester, shell=/bin/bash, from=/dev/pts/1
```

Figure 3.15: creation of a new user alert

- Creation of a new group named “pentester”, as illustrated in figure 3.16:

```
rabebe-virtual-machine useradd[8005]: new group: name=pentester, GID=1002
```

Figure 3.16: Creation of a new group alert

As part of the alerting mechanism, Splunk was configured to send alert notifications to the system administrator through the Splunk Mobile App as displayed in the following figure 3.17:

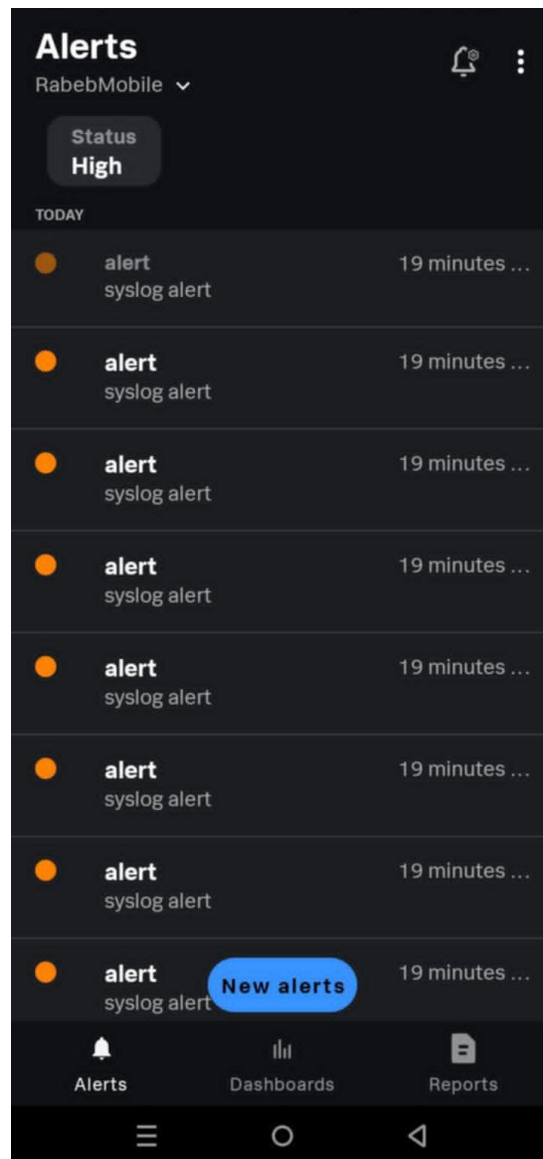


Figure 3.17: Splunk Mobile App alerts

3.5 Prevention

After detecting a brute-force SSH attack on the client system, it became important to establish a comprehensive security policy to prevent such attacks. Brute-force attacks attempt to acquire unauthorized access by systematically guessing user credentials, posing a significant danger to system integrity. To address this, several preventative steps were implemented, including securing the SSH service, removing potential attacker access, and hardening the system settings. The following approach summarizes the actions taken to reduce the danger and strengthen the system's resilience to future attacks.

3.5.1 Password strengthening

The first step in the prevention strategy is to change the user password to a strong, secure password, as seen in figure 3.18 below. This helps eliminate the risk of compromised or weak credentials being used by attackers to gain unauthorized access.

```
rabeb@rabeb-virtual-machine:~$ sudo passwd rabeb
New password:
Retype new password:
passwd: password updated successfully
```

Figure 3.18: Change password

3.5.2 User account audit and cleanup

After securing passwords, the next critical step involved reviewing all existing user accounts on the system as displayed in the following figure 3.19. This process aimed to identify any accounts that may have been created or modified by an attacker during the brute-force attempt.

```
rabeb@rabeb-virtual-machine:~$ sudo grep '/home' /etc/passwd | tail -n 5
[sudo] password for rabeb:
syslog:x:104:111::/home/syslog:/usr/sbin/nologin
cups-pk-helper:x:115:122:user for cups-pk-helper service,,,:/home/cups-pk-helper
:/usr/sbin/nologin
rabeb:x:1000:1000:rabeb,,,:/home/rabeb:/bin/bash
pentester:x:1002:1002::/home/pentester:/bin/bash
```

Figure 3.19: Reviewing user accounts

During the audit, a suspicious user account named "pentester" was discovered. This account was not part of the system's original configuration and was most likely created by the attacker after the brute-force attempt. To prevent further exploitation or privilege escalation, the "pentester" account was immediately deleted from the system as shown in figure 3.20:

```
rabeb@rabeb-virtual-machine:~$ sudo userdel -r pentester
userdel: pentester mail spool (/var/mail/pentester) not found
```

Figure 3.20: Delete suspicious users

3.5.3 IP blocking

Following the detection of the brute force attack, log analysis was conducted to identify the source of the malicious SSH login attempts. The attacker's IP address was successfully extracted from the logs as evidenced in figure 3.21:

```
rabeb-virtual-machine sshd[4617]: Failed password for rabeb from 192.168.59.130 port 60070 ssh2
rc = syslog : source type = login
```

Figure 3.21: Extract attacker IP address

To prevent any further unauthorized access attempts from this source, the IP address was promptly blocked at the firewall level as shown in figure 3.22. This action effectively cut off the attacker's ability to reach the system and serves as an immediate containment measure while longer-term protections are implemented.

```
rabeb@rabeb-virtual-machine:~$ sudo ufw deny from 192.168.59.130
Rules updated
```

Figure 3.22: Block attacker IP address

3.5.4 SSH configuration hardening and port modification

To reduce the system's exposure to brute-force attacks and unauthorized access, the SSH configuration was reviewed and hardened. Root login was explicitly disabled to prevent attackers from directly targeting the root account as illustrated in figure 3.23:

```
# the setting of "PermitRootLogin without-password".
PermitRootLogin no
```

Figure 3.23: Disable SSH root login

Also, password-based authentication was turned off as shown in figure 3.24:

```
# To disable tunneled clear text passwords, change to no here!
PasswordAuthentication no
```

Figure 3.24: Turn off password-based authentication

Additionally, SSH access was restricted to key-based authentication only as illustrated in figure 3.25. This ensures that only users with valid cryptographic keys can establish an SSH connection, significantly increasing the difficulty of unauthorized access through credential guessing or brute-force techniques.

```
PubkeyAuthentication yes
```

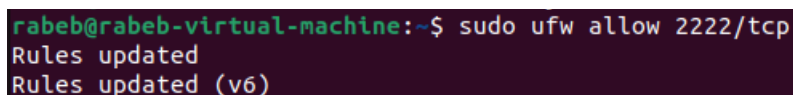
Figure 3.25: Enable key-based authentication

As a further preventive measure, the default SSH port 22 was changed to a non-standard port 2222, as indicated in figure 3.26, to reduce visibility to automated scanning tools commonly used in brute-force attacks.

```
Port 2222
#AddressFamily any
#ListenAddress 0.0.0.0
#ListenAddress ::
```

Figure 3.26: Change SSH port

Corresponding in figure 3.27, the firewall rules were updated to allow traffic on the new SSH port.

A terminal window with a dark background. The prompt is 'rabeb@rabeb-virtual-machine:~\$'. The command entered is 'sudo ufw allow 2222/tcp'. The output shows 'Rules updated' on two lines.

```
rabeb@rabeb-virtual-machine:~$ sudo ufw allow 2222/tcp
Rules updated
Rules updated (v6)
```

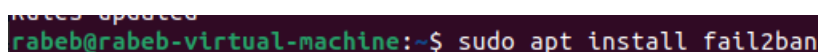
Figure 3.27: Allow traffic on the new SSH port

3.5.5 Fail2ban Deployment

Fail2Ban is an intrusion prevention system that can block attackers from accessing your server. It can be thought of as an automated bouncer that watches your server's log files for suspicious activity and kicks out bad actors before they can cause trouble.

It constantly scans your server's logs, looking for anyone repeatedly trying to guess passwords or force their way in. While it started as a simple SSH protector, Fail2Ban can be configured to monitor almost everything on your server – from your website traffic to email services and file transfer system [27].

To enhance automated protection against repeated unauthorized access attempts, Fail2ban was installed and configured on the client system as demonstrated in figure 3.28:

A terminal window with a dark background. The prompt is 'rabeb@rabeb-virtual-machine:~\$'. The command entered is 'sudo apt install fail2ban'. The output shows 'Rules updated' on the line above and 'fail2ban' on the line below.

```
Rules updated
rabeb@rabeb-virtual-machine:~$ sudo apt install fail2ban
fail2ban
```

Figure 3.28: Fail2ban installation

Fail2ban regularly monitors system logs, particularly SSH authentication logs, for patterns of brute-force behavior, such as many failed login attempts within a brief duration of time. When such activity is detected, the offending IP address is automatically blocked for a set period, essentially preventing the attacker from gaining network access.

3.6 Conclusion

This chapter demonstrated the effectiveness of the security monitoring and response setup through the simulation of an attack strategy. The test validated each phase of the process: the execution of the attack, the detection and alerting by the SIEM solution, and the implementation of appropriate prevention measures. The system successfully identified suspicious activity, generated alerts in real-time, and allowed for a prompt and structured response.

Preventive actions such as securing user accounts, blocking malicious IP addresses, hardening SSH configurations, updating firewall rules, and deploying automated protection tools like Fail2ban were applied to strengthen the system's defenses. These results confirm that the current architecture is capable of detecting, responding to, and mitigating similar threats in a real-world environment.

General conclusion and perspectives

In conclusion, this SIEM project highlighted the critical need for centralized and resilient security monitoring to effectively identify and respond to cyberattacks in real time. By simulating various attack scenarios such as brute-force SSH intrusions and capturing their traces using Splunk, the project demonstrated how comprehensive log collection, correlation, and analysis can expose malicious behaviors that may otherwise go undetected. The use of Splunk's powerful interface for threat visualization, alert generation, and incident investigation significantly enhanced the ability to respond swiftly and accurately.

To ensure high availability and data redundancy, the SIEM environment was designed using Splunk indexer clustering, with multiple indexers and a cluster master to support log replication and fault tolerance. This architecture guarantees that log data remains accessible and protected even in the event of hardware failure, ensuring continuous monitoring and operational reliability.

The prevention phase of the project focused on implementing essential system hardening and threat mitigation techniques. These included securing user accounts by auditing and removing unauthorized users, changing passwords, blocking attacker IP addresses, modifying SSH configurations to enforce key-based authentication, changing the default SSH port, updating firewall rules, and deploying Fail2ban to automatically detect and block brute-force attempts. These actions not only neutralized active threats but also significantly improved the system's defensive posture.

Looking ahead, the integration of artificial intelligence and machine learning into SIEM systems is essential for building more adaptive cybersecurity solutions. Traditional rule-based detection is limited to known threats and often fails to detect advanced attacks like zero-day exploits. AI enables SIEM platforms to learn normal system behavior and identify anomalies in real time. This improves detection accuracy, reduces false positives, and accelerates incident response. Ultimately, AI transforms SIEM from a reactive system into a proactive and intelligent defense mechanism.

From a personal perspective, this project allowed me to develop concrete skills in cybersecurity, system administration, log management, and the practical use of a SIEM platform. It also strengthened my ability to manage a project from design to deployment, while applying good practices in threat detection and prevention.

Bibliography

- [1] Centre National de l'Informatique. (n.d.). *Who we are*.
<http://www.cni.tn/index.php/en/layout-4/who-we-are/>
- [2] IBM. (n.d.). *SIEM – Security Information and Event Management*.
<https://www.ibm.com/think/topics/siem>
- [3] Exabeam Security Team. (n.d.). *The Essential Guide to SIEM*. Exabeam.
- [4] OpenText. (2023, February 27). *SIM, SEM and SIEM: Definitions and choosing the right enterprise solution*. <https://community.opentext.com/cybersec/b/cybersecurity-blog/posts/sim-sem-and-siem-definitions-and-choosing-the-right-enterprise-solution>
- [5] ResearchGate. (n.d.). *High-level SIEM Architecture* [Figure].
https://www.researchgate.net/figure/High-level-SIEM-Architecture_fig1_384271411
- [6] Last9. (2023, September 18). *SIEM Architecture: Core components & high-level flow*.
<https://last9.io/blog/siem-architecture/>
- [7] Microsoft. (n.d.). *What is SIEM?* <https://www.microsoft.com/en-gb/security/business/security-101/what-is-siem#:~:text=SIEM%20tools%20collect%2C%20aggregate%2C%20and,define%20threats%20and%20generate%20alerts>.
- [8] Exabeam. (n.d.). *SIEM Solutions*. <https://www.exabeam.com/explainers/siem-tools/siem-solutions/>
- [9] Suricata. (n.d.). *Official Website*. <https://suricata.io/>
- [10] Paessler AG. (n.d.). *What is Syslog?* <https://www.paessler.com/it-explained/syslog>
- [11] Comodo. (n.d.). *Is Splunk a SIEM?* <https://www.comodo.com/is-splunk-a-siem.php>
- [13] Wikimedia Commons. (n.d.). *Splunk logo* [Image].
https://commons.wikimedia.org/wiki/File:Splunk_logo.png

- [14] Splunk Inc. (n.d.). *Splunk's architecture and what gets installed*.
<https://docs.splunk.com/Documentation/Splunk/9.4.0/Installation/Splunksarchitectureandwhatgetsinstalled>
- [15] TechTarget. (n.d.). *VMware Workstation*.
<https://www.techtarget.com/searchvmware/definition/VMware#:~:text=VMware%20Workstation%20is%20the%20first,such%20as%20Windows%20or%20Linux.>
- [16] Wikimedia. (n.d.). *VMware Workstation 16 icon* [Image].
https://en.m.wikipedia.org/wiki/File:Vmware_workstation_16_icon.svg
- [17] Docker Hub. (n.d.). *Ubuntu image*. https://hub.docker.com/_/ubuntu/
- [18] 1000logos.net. (n.d.). *Ubuntu logo*. <https://1000logos.net/ubuntu-logo/>
- [19] Kali Linux. (n.d.). *What is Kali Linux?* <https://www.kali.org/docs/introduction/what-is-kali-linux/>
- [20] Pinclipart. (n.d.). *Kali Linux logo PNG* [Image].
https://www.pinclipart.com/downpngs/TxomxJ_kali-linux-kali-linux-logo-png-clipart/
- [21] UpGuard. (2023, May 15). *How to use Nmap*. <https://www.upguard.com/blog/how-to-use-nmap#toc-0>
- [22] Nmap. (n.d.). *Nmap official images*. <https://nmap.org/images/>
- [23] Linux Magazine. (n.d.). *Scanning with Zenmap*. <https://www.linux-magazine.com/Online/Features/Scanning-with-Zenmap>
- [24] ManageEngine. (n.d.). *Detecting Hydra using correlation rules*.
<https://www.manageengine.com/log-management/correlation-rules/detecting-hydra.html>
- [25] PNGEgg. (n.d.). *Hydra icon PNG* [Image]. <https://www.pngegg.com/en/png-pkxmr/download>
- [26] Splunk Documentation. (n.d.). *About indexer clusters*.
<https://docs.splunk.com/Documentation/Splunk/latest/Indexer/Aboutclusters>
- [27] RunCloud. (n.d.). *What is Fail2Ban?* <https://runcloud.io/blog/what-is-fail2ban>
- [28] IBM. (n.d.). *Intrusion detection system*. <https://www.ibm.com/think/topics/intrusion->

detection-system

[29] GeeksforGeeks. (n.d.). Intrusion Detection System (IDS).

<https://www.geeksforgeeks.org/intrusion-detection-system-ids/>

[30] Logit.io. (n.d.). Top 6 tips for forwarding logs. <https://logit.io/blog/post/top-6-tips-for-forwarding-logs/>